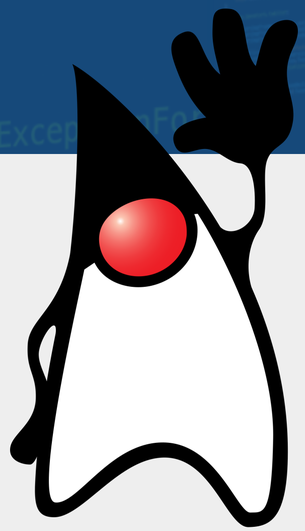
Java Programlama Dili

1. Bölüm

Java Geliştirme Ortamı

Akın Kaldıroğlu

BTK AKADEMİ



Konular - I



- **Ön Açıklama**
- **Platformlar**
- **"Java" ile Neyi Kastediyoruz?**
- **JDK, JRE ve JVM**
- **Makinemde Java Var mı?**
- **Java Dağıtımları**
  - OpenJDK
  - Oracle JDK
  - Diğer Dağıtımlar
  - Hangisini Kurayım?
- **Java Ayarları**
- **JDK Kurulumu**
- **MS Windows'ta JDK Kurulumu**
  - MS Windows'ta Oracle JDK Kurulumu
  - MS Windows'ta PATH ve JAVA_HOME Ayarları
  - MS Windows'ta Temurin ve Microsoft Build of OpenJDK Kurulumu

Konular - II



- **macOS'ta JDK Kurulumu**
  - macOS'ta PATH ve JAVA_HOME Ayarları
- **Linux'ta JDK Kurulumu**
- **Birden Fazla JDK**
- **JShell**
  - JShell Komutları
- **Geliştirme Ortamları**
  - IntelliJ IDEA
  - Dersin Kaynak Kodları
  - Yeni Proje Oluşturma
- **Ödevler**
- **Bundan Sonra?**

İçerik



- Bu bölümde Java geliştirme ortamı (Java development environment) kısaca tanınmaya başlanacak, ana terimleri, geliştirme araçları ve teknolojileri incelenecektir
- Microsoft Windows, Apple macOS ve Ubuntu Linux üzerinden Java geliştirme ortamının hazır hale getirilmesi detaylı bir şekilde ele alınacaktır
- Bunu yaparken Java programlarına ve kod yapısına aşina hale gelmek ve eğlenmek için ufak tefek programlar da çalıştırılacaktır

Ön Açıklama



Dil Meselesi - I



- Türkçeleri yaygın kullanılan terimleri ben de Türkçe olarak kullandım ama her halükarda İngilizcelerini de belirttim: *Kurulum* (*installation*), *sürüm* (*version*, *release*) gibi
- İngilizcesi yaygın olan terimleri aslındaki gibi kullandım: *Java Development Kit* (*JDK*) ya da *Java Virtual Machine* (*JVM*) gibi
  - Bazen, İngilizcesi yaygın olan terimlerin yaygın olmasa bile, anlaşılmayı güçlendirmek adına, makul Türkçelerini teklif ettim: *encapsulation* (*sarmalama*), *dependency* (*bağımlılık*), *Java Virtual Machine* (*Java Sanal Makinası*)

Dil Meselesi - II



- Bazı Türkçe terimler ve karşılıkları da şöyledir:
  - Dizin = Directory
  - Dosya = File
  - Patika, Yol = Path
  - Varsayılan = Default
  - Derleyici = Compiler
  - Araç = Tool
  - Çalışma zamanı = Run-time
  - Seçenek, Parametre = Flag

Uygulamalar ve Ödevler



- Eğitimimizde genelde kodlar üzerinden yapacağımız uygulamalar ile size yapmak üzere bıraktığım ödevler olacak
- Ödevler, araştırmalar yanında genelde derste öğrendiğimiz konuları uygulayacağınız hatta muhtemelen derste ele aldığımız kodlara benzer kodlar yazacağınız çalışmalardır
- Lütfen ödevleri yapın, yapıncaya kadar uğraşın!
- Yapamazsanız dersi tekrar dinleyin, dersteki kodları tekrar çalışın sonra ödevi tekrar yapmaya girişin ve mutlaka yapın

Kaynak Kodlar Hakkında - I



- Derste kullandığım kodları bu eğitim için hazırladım ve sizinle paylaştım
- Zaman zaman derste ben de kod yazacağım, belki kendi kendime konuşur şekilde, bazı örnek programları kodlayacağım
- Ama derste kullanacağım kodların hepsinin ders sırasında yazılması mümkün değildir, bu eğitimin süresini 3-5 katına çıkarır
- Zaten buna gerek de yoktur, çünkü her kodun önceden yazılsa bile ne yaptığını, satır-satır detaylıca açıklayacağım

Kaynak Kodlar Hakkında - II

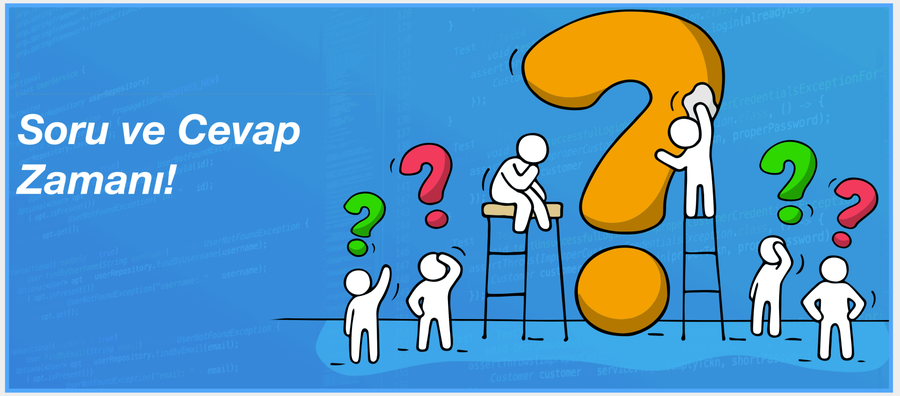


- Gerçek dünyada devamlı yazılmış kodu okursunuz ve anlamaya çalışırsınız
- Kod okumak, AI çağında daha da yoğun olarak iş hayatımızda yer alıyor
- Ama unutmayın ki kod okuyabilmek, kod yazmış olmayı gerektiriyor

Sorularınız



- Konular arasında soru sayfaları var
- Bana soracağınız soruları, daima bu soru sayfasının sonunda olduğu konu ismiyle sorun, mümkünse sayfa numarasını da belirtin
- Yani, "*hocam, 'Java ile Neyi kastediyoruz' kısmında aklıma takıldı da …*" ya da "*'JDK, JRE ve JVM' ile ilgili bir sorum var…"* şeklinde bana soru sorabilirsiniz



Soru ve Cevap Zamanı!





Platformlar



Platform - I



- Bilgisayar donanımı (hardware) ile üzerinde çalışan işletim sisteminden (İS) (operating system (OS)) oluşan yapıya **platform** denir

Platform = Donanım (HW) + İS (OS)

- i7 gibi Intel CPU mimarisi (x64) üzerinde çalışan Windows (Wintel), macOS (MacIntel) ya da Linux işletim sistemleri farklı birer platformdur

- Hatta aynı CPU üzerinde çalışan Windows NT, Windows XP, Windows Vista, Windows 7, Windows 8, Windows 10 , Windows 11 vd. hepsi ayrı birer platformdur

Platform - II



- Windows 11 ya da macOS Golden Gate, Intel CPU'ları üzerinde çalışabileceği gibi ARM mimarisine sahip işlemciler üzerinde de çalışabilir
  - Bu iki CPU mimarisi genelde X64 ve AArch64 (ya da ARM64) olarak ifade edilir
- Örneğin ben daha önce Intel i5, i7 ve i9 üzerinde çalışan farklı macOS İS'leri kullandım ama şimdi ARM64 mimarisine sahip M3 CPU'lu bir macOS kullanıyorum

Platform - III



- Platformlar 32 bit ya da 64 bit olabilir
  - Bu ayrım, Java ve tüm yazılımlar ile ilgili kurulumlarda (installation) yanlış yapmamak için önemlidir, dolayısıyla platformunuzun 32/64 bitlik olduğunu daima bilin
  - Öte yandan artık üretilen hemen hemen tüm platformlar 64 bitliktir, 32 bitlik platformlar tarihte kalmıştır
- 32 bitlik Java Windows için 24., Linux için 25. sürümle tamamen kaldırıldı; Java 25 yalnız 64 bitliktir

Farklı Sürümler ve Paketler



- Platformların sürümleri arasında Java açısından zaman zaman ufak-tefek farklılıklar olabileceğini daima göz önünde bulundurun
- Örneğin Linux platformları genelde birbirlerinden farklılık gösterir
  - Bu derste böyle durumlar karşılaşıldıkça belirtilecektir
- Dolayısıyla platformunuz için geçerli olan kurulum (installation) ve ayarları (settings) bilmeniz ve yapmanız gereklidir

Komut Satırı



- Bu derste sıklıkla **komut satırı**nı (**command line**) kullanacağız
- Komut satırı ile temelde
  - Microsoft Windows'ta (ya da kısaca MS Windows ya da Windows) **`CMD`**, (**`Command Prompt`**)
  - macOS ve Linux'ta ise **`terminal`** denen programlar kastedilmektedir
- Aynı işi yapan *Warp*, *iTerm*, *Xterm* vb. alternatif araçlar da vardır

Genel Uyarı



- Platformlarınızda kullandığınız dosya ya da dizin patikalarında (ya da yollarında) (path) daima İngilizce karakterler kullanın; Türkçe ve `'%#<>"!` gibi özel karekterlerden kaçının, bu tür karakterler sıklıkla erişim hataları verir
  - Windows sistemlerde patikalarda boşluk kullanmak da problem yaratabilir
- Biz bu derste çok özel bir ihtiyaç olmadıkça kodlarımızda hiçbir Türkçe veya özel karakter içeren isim kullanmayacağız

Benim Platformlarım



- Ben Apple MacBook Pro M3 Max üzerinde macOS Tahoe kullanıyorum
- Makinemde VirtualBox (https://www.virtualbox.org/) üzerinde çalışan Ubuntu mevcut
- Ayrıca gerektikçe kullandığım bir Microsoft Windows 10 makinam da var
- Java'nın kurulumlarını ve ayarlarını size bu üç platform üzerinde göstereceğim

Soru ve Cevap Zamanı!





"Java" ile Neyi Kastediyoruz?



Java'nın Paketleri



- Java'nın temelde 2 farklı tür uygulamaya yönelik 2 farklı paketi vardır:
  - **Standart Java** (**Standard Edition, SE**), bu eğitimde ele alınacak olan çekirdek dildir (core language), **Temel Java** olarak da adlandıracağız
  - **Kurumsal Java** (**Jakarta EE, JEE,** (eski ismi **Enterprise Edition, EE**)), kurumsal uygulamalar için gerekli yapıları barındırır
- Her birinin ayrı sürümleri (version/release) vardır
- Eskiden gömülü ve mobil uygulamalar için geliştirilmiş olan **Micro Edition**'ın artık sadece bakımı yapılıyor, yeni sürümü çıkarılmıyor

Java Derken Neyi Kastediyoruz?



- Biz bu eğitimde "Java" dediğimiz zaman çoğunlukla Temel Java'yı yani Java SE'yi kastedeceğiz
- Bazen de daha geniş bir anlamda, her türlü Java uygulamasını, Java'nın, temel olsun kurumsal olsun, her türlü kullanımını kastedeceğiz
- Bunu bağlamdan anlayacaksınız

Java SE'nin Sürümleri - I



- Java SE'nin şu anda Eylül 2026'da çıkmış olan 27. sürümü (version/release) vardır
- 2018'deki 10. sürüm öncesinde genelde 3-4 senelik aralıklarla yeni Java sürümü çıkıyordu ve biz yeni sürümü özlüyorduk 😀
- 10. sürümle bu plan değişti ve artık Java, küçük içerikli, genelde Mart ve Eylül aylarında senede iki defa çıkan sürümlerle gelişir oldu
- Örneğin Mart 2027'de 28. sürüm gelecektir; hatta siz bu eğitime ulaştığınızda muhtemelen daha yeni sürümler de çıkmış olacaktır

Java SE'nin Sürümleri - II



- Ayrıca Java için ana ve ara sürümler de söz konusudur:
  - Ana sürümler (major version) uzun süreli desteğe (**Long-Term-Support**, **LTS**) sahiptir ama ara sürümlerin (minor version) 6 ay sonra yenisi gelince desteği devam etmez; dolayısıyla projelerde daima ana sürümler kullanılır
- Eylül 2025 itibarıyla en son ana sürümler ve çıkış yılları şöyledir:
  - 8: 2014, 11: 2018, 17: 2021, 21: 2023, 25: 2025
- Java geliştiricileri yeni ana sürümlere geçmektedir ama sektörde 8 öncesi sürümlere de sıklıkla rastlanır

Java SE'nin Sürümleri - III



- En son çıkan ana sürüm yani Long-Term-Support'a sahip sürüm 25'tir
- Ayrıca sürümlerin 25.0.2 gibi, bug fix amaçlı güncellemeleri (update) yayınlanır
  - Bu güncellemeler ara sürümlerde sadece 6 ay için, bir sonraki sürüme kadar yapılırken ana sürümlerde çok daha uzun sürelerle yapılır
    - Bu yüzden ana sürümlere Long-Term-Support (LTS) denir
  - Bir sonraki LTS sürümünün 29 olması planlanmıştır

Java SE'nin Sürümleri - IV



- Bu bahsedilen sürümler OpenJDK'nın sürümleridir
  - OpenJDK birazdan ele alınacaktır
- Java'nın gelecekteki sürüm planı için şu adrese bakabilirsiniz:
  - https://www.oracle.com/java/technologies/java-se-support-roadmap.html

Eğitimde Kullanılacak Sürüm



- **Biz bu eğitimde 16 Eylül 2025'te yayınlanan 25. ana sürümü temel alacağız**
- Dolayısıyla bu sürümle gelen yenilikler de konularımızda bulunacaktır
- Ama bahsettiğim bu 3 platform için şu anda var ya da gelecekte çıkacak olan Java sürümlerini kurmayı ve ayarlarını yapmayı öğrendiğinizde hangi sürüm olursa olsun geliştirme ortamınızı hazır hale getirebileceksiniz

Soru ve Cevap Zamanı!





JDK, JRE ve JVM



JDK - I



- **JDK**, **Java Development Kit**'in kısaltmasıdır
  - Java SDK (Software Development Kit) de denir
- Java ile kod geliştirmek için gerekli yazılım geliştirme kitidir
  - Kit: Malzeme/alet takımı, çantası
- Java kodlarını derlemek (compilation), debug etmek, çalıştırmak (run) vb. işler için gerekli araçları (tool) ve bu araçları çalıştırmak için kullanılacak komutları (command) içerir

JDK - II



- JDK'da bulunan araçların (çalıştırılabilirlerin (executables ya da binaries) de denir) yani komutların (commands) en önemlileri şunlardır:
  - **javac**: Java derleyicisidir (compiler), Java kaynak kodlarını derler
  - **java**: Java yorumlayıcısıdır (interpreter), Java'nın çalışma zamanı ortamını (Java Runtime Environment, JRE) başlatır ve Java kodlarını çalıştırır
  - **jdb**: Java debuggerdır
  - **jshell**: JShell komutudur, bu bölümün sonunda detaylarını öğreneceğiz

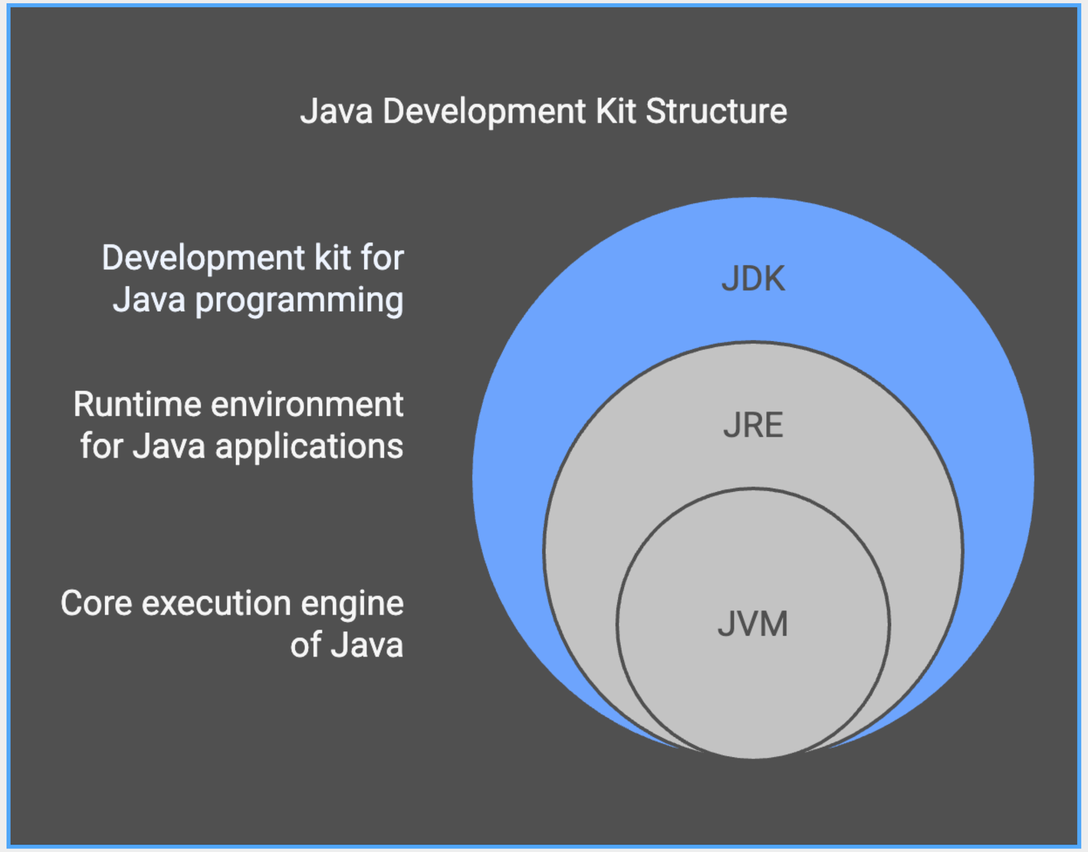
JRE



- **JRE**, **Java Runtime Environment**'in kısaltmasıdır
- JRE, Java'nın hem çalışma zamanı yapılarını (kurulum, dosyalar vs.) hem de çalışma zamanının kendisini temsil eder
  - **Çalışma zamanı** ya da **run-time**, programın çalışmakta olduğu zamanı, bu zamanda bellek (memory) vb. kaynakları ve ortamı ifade eder
- JRE, JVM ve Java kütüphanelerini (Java class library) barındırır
- JRE, `java` komutuyla başlatılır

JDK ve JRE - I

- **Java'da geliştirme yapmak için JDK, Java uygulamalarının çalışması için JRE gereklidir**
- Tipik olarak yazılım geliştiricilerin makinelerinde JDK, Java programlarının çalıştığı sunucu (server) makinalarında ya da tarayıcı, saat, telefon, araba, elektrikli ev eşyası vb. ortamlarda ise JRE bulunur

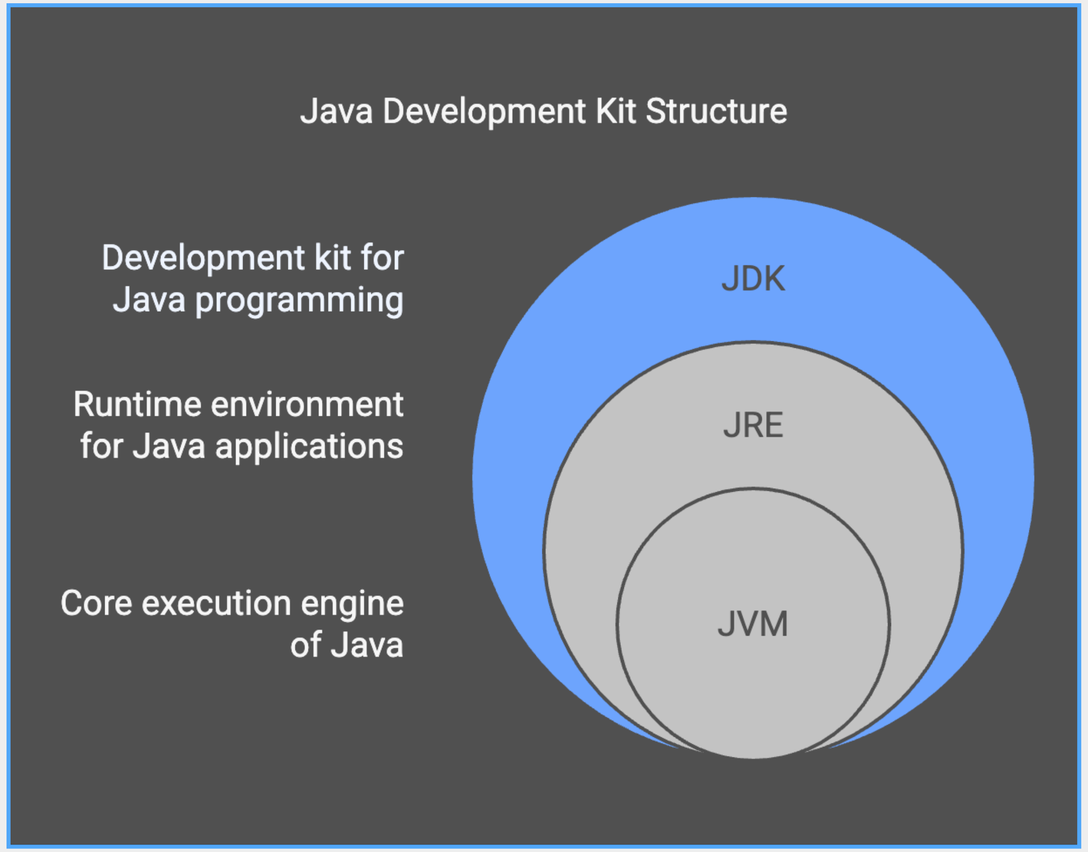


napkin.ai

JDK ve JRE - II



- JDK, JRE'yi içerir ama JRE, JDK'yı içermez
- JDK ve JRE'nin, MS Windows, macOS ve Linux gibi pek çok platform için dağıtımları (distribution) vardır
- Birazdan bunları detaylıca ele alacağız



napkin.ai

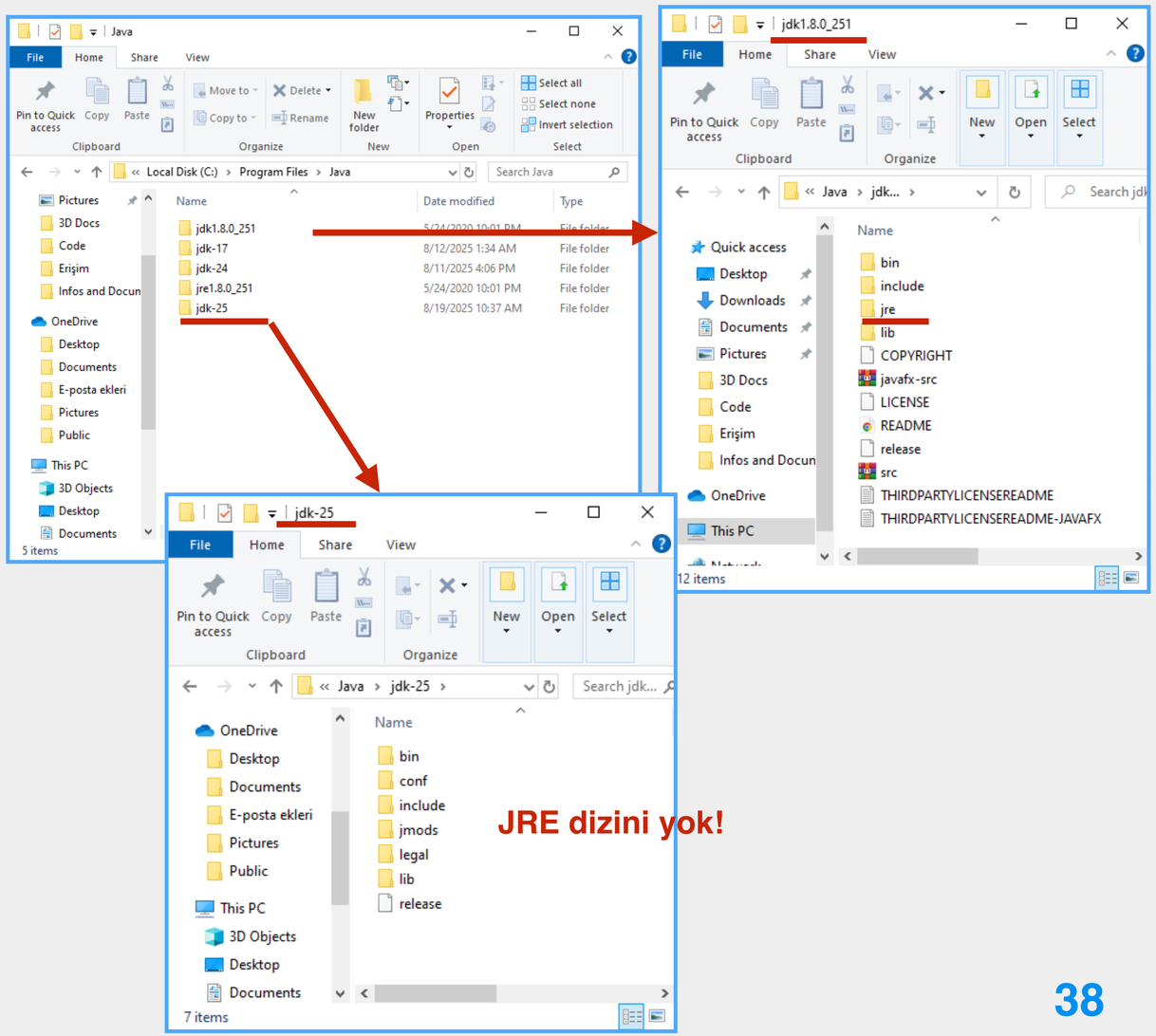
JDK ve JRE - III



- Java 11'e kadar JRE, Oracle ve OpenJDK tarafından ayrıca indirilip kurulabilir bir dağıtım olarak sunulmasına rağmen bu sürümden itibaren artık ayrı JRE dağıtımı sunulmamaktadır
- Dolayısıyla Oracle JDK ve OpenJDK için artık JRE ayrı bir kurulum ya da dizin yapısını değil, çok daha sanal bir kavram olarak, Java'nın çalışma ortamını (runtime environment) temsil etmektedir

JDK ve JRE - IV

- Örneğin, Windows makinemde hem JDK 1.8 hem de JRE 1.8 kuruludur
- Ama 11. sürümden sonra artık genelde sadece JDK kurulumları söz konusudur
- Bu sebeple bir makinada JDK 25 varken JRE 25 ancak JDK 25'ten elde edilebilir



JDK ve JRE - V



- Ama örneğin OpenJDK'nın Eclipse Temurin dağıtımı hala ayrı bir JRE paketi sunmaktadır
- OpenJDK ve Oracle JDK için de isteyenler `jlink` isimli bir JDK aracı yardımıyla, JDK'nın daha küçük paketlerini, yani kendi uygulamalarına özel JRE'lerini oluşturabilir

JVM



- **JVM**: **Java Virtual Machine**'in (**Java Sanal Makinesi**) kısaltmasıdır ve derlenmiş Java kodlarının üzerinde çalıştığı yazılımdır, sanal bir makinedir
- JVM, İS üzerinde çalışan, Java'nın çalışma zamanı işlemidir (process)
- JVM, Java kodlarını platformdan yalıtan, son derece karmaşık, CPU ve bellek gibi kaynakların kullanımını yöneten bir programdır
- JVM, JDK ve JRE'nin bir parçası olarak bulunur

Sınıf Kütüphanesi



- JRE, tekrar kullanılabilen (reusable) bir **sınıf kütüphanesi** (**class library**) de içerir
- Sınıf kütüphanesi, Java ile geliştirilmiş ve derlenmiş sınıf (class) vb. yapılardan oluşur
- **`String`**, **`Date`** vb. pek çok hazır sınıfı kullanıma sunar
- Bu hazır yapılar Java'da kod yazmayı hızlandırır

Soru ve Cevap Zamanı!







Uygulama

İlk Kod



- Programlama öğreten İngilizce kitaplarda ilk kod örneği, eskiden bu yana daima **`Hello World`**'dür
- Biz de şimdi **`Selam :)`** yazan birkaç farklı kodu yazıp çalıştıracağız
- Aynı zamanda **`java`** komutunun uygulanmasını göreceğiz
  - Tabii olarak bu komutun JDK 25 ile gelen halini kullanacağız
- Bundan emin olmak için komut satırında **`java -version`** çalıştırabilirsiniz (bunu birazdan detaylı öğreneceğiz)

Selam :)



- Java'daki ilk kodumuz, mümkün olan en basit Java kodudur ve bize selam vermektedir 😍
- Çok detaylara takılmadan kodu çalıştıralım ve keyfini çıkaralım

```
void main() {
    System.out.println("Selam :)");
}
```

Selam24.java - I



- Şimdi makinanızda, masa üstünde (desktop) **`Code`** isimli bir yeni dizin yaratın ve içine **`ch01`** isimli başka bir dizin koyun
- Sonra aşağıdaki kodu kopyalayıp yapıştırarak ya da aynısını yazarak **`ch01`** dizininde **`Selam24.java`** isimli bir dosya yaratın

```
macOS ⇒ /Users/akin/Desktop/Code/ch01/Selam24.java
Windows ⇒ C:\Users\kaldi\Desktop\Code\ch01\Selam24.java
Ubuntu ⇒ /home/kaldiroglu/Desktop/Code/ch01/Selam24.java
```

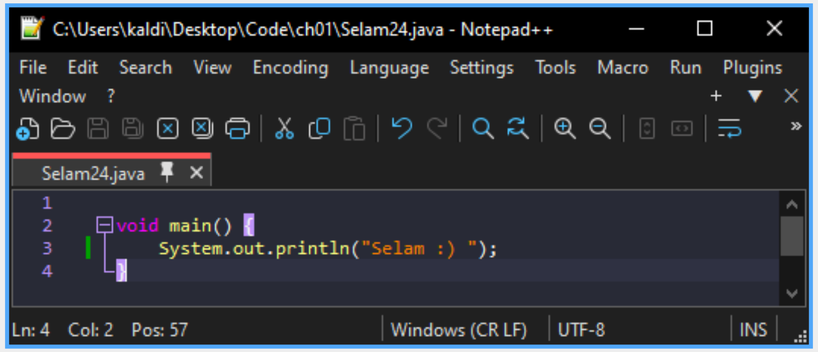
```
void main() {
    System.out.println("Selam :)");
}
```

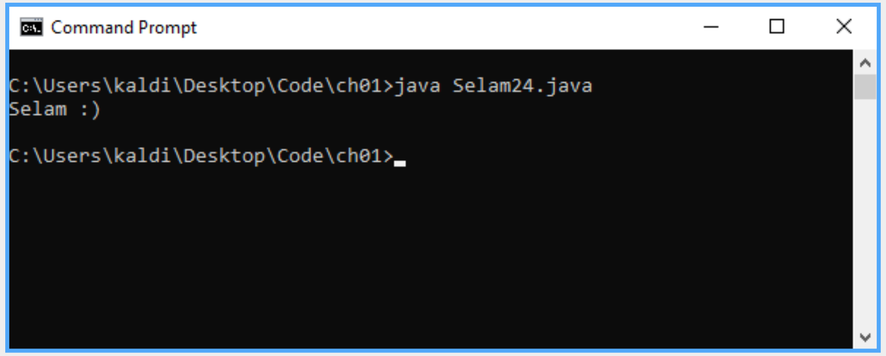
Selam24.java - II



- Daha sonra komut satırında **`ch01`** dizinine gidip **`java Selam24.java`** komutuyla ilk kodumuzu çalıştırın
- **`java`**, Java kodunu çalıştıran komuttur; bu komutla JVM, **`Selam24.java`** kodunu yükler ve çalıştırmaya başlar ve komut satırında **`Selam :)`**'ı görürüz

```
void main() {
    System.out.println("Selam :)");
}
```



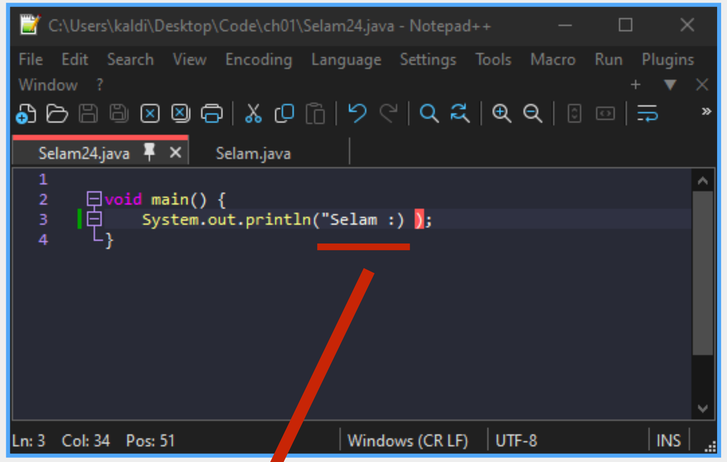


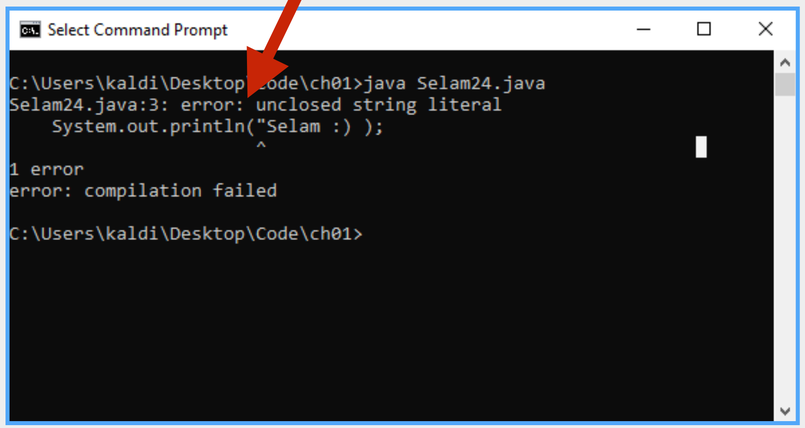
Selam24.java - III



- Java'nın kurallarını ihlal eden kod yazarsanız, örneğin bir **`String`** olan **`Selam :)`** değerini iki tane çift tırnak içine almazsanız, hata alırsınız
- Dolayısıyla kodu doğru yazdığınızdan emin olun, hata alırsanız hatayı anlayarak kodunuzu düzeltin ve tekrar çalıştırın

```
void main() {
    System.out.println("Selam :)");
}
```





Selam25.java



- Bu Java kaynak kod dosyasının ismine 24'ü, Java'nın 24. sürümüyle gelen özelliklerini kullandığımı belirtmek için koydum
- Aynı kodu Java'nın 25. sürümüyle gelen ve detaylarını sonra göreceğimiz özelliklerle, daha basit haliyle yazarsak şöyle olur:

```
void main() {
    IO.println("Selam :)");
}
```

- Bu kodu **`Selam25.java`** dosyasına kaydedin, **`java Selam25.java`** ile çalıştırın ve aynı çıktıyı görün

Selam.java



- Bu kodun, Java'nın ilk sürümüyle bile çalışacak en klasik hali şöyledir

```
class Selam{
   public static void main(String[] args) {
    System.out.println("Selam :)");
   }
}
```

- Bu kodu da **`Selam.java`** dosyasına kaydedin ve benzer şekilde **`java Selam.java`** komutuyla çalıştırın
  - Kodu, IDE'deki dosyadan alıyorsanız, başındaki **`package`** ile başlayan satırı almayın çünkü henüz **`package`** ile nasıl çalıştıracağımızı bilmiyoruz

Selam Programları - I



- Java'nın 25 sürümündeki **`java`** komutuyla bu 3 farklı **`Selam`** uygulaması çalıştırılabilir
- JDK 24 öncesi bir sürümle sadece ilk, klasik hali çalıştırılabilir
- 3. halini ancak Java 25 ve daha yeni sürümleriyle çalıştırabiliriz çünkü en yeni özellikleri kullanmaktadır

1

```
class Selam{
   public static void main(String[] args){
    System.out.println("Selam :)");
   }
}
```

2

```
void main() {
    System.out.println("Selam :)");
}
```

3

```
void main() {
    IO.println("Selam :)");
}
```

Selam Programları - II



- Görüldüğü gibi, Java'nın son sürümlerinde, Java'yı öğrenenlere kolaylık amacıyla bazı basitleştirmeler yapılmıştır

1

```
class Selam{
   public static void main(String[] args){
    System.out.println("Selam :)");
   }
}
```

2

```
void main() {
    System.out.println("Selam :)");
}
```

3

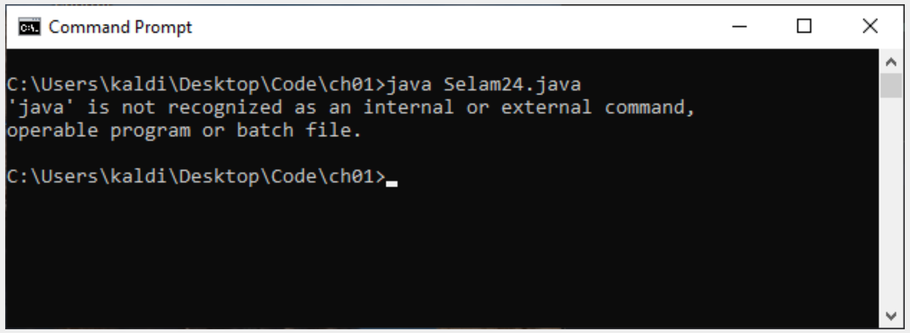
```
void main() {
    IO.println("Selam :)");
}
```

Hata Durumunda



- Kodları çalıştırırken **`java`** komutu bulunamadığı gerekçesiyle hata alırsanız dert etmeyin, bu henüz makinanızda Java geliştirme ortamı olmadığı ya da olsa bile gerekli ayarlar yapılmadığı içindir

- Örnek **`Selam`** kodlarından bazısını çalıştırıp bazısını çalıştıramazsanız, bu makinanızda en son 25. sürümün değil önceki sürümlerin var olduğu anlamına gelir



Gözden Geçirme Özellikleri - I

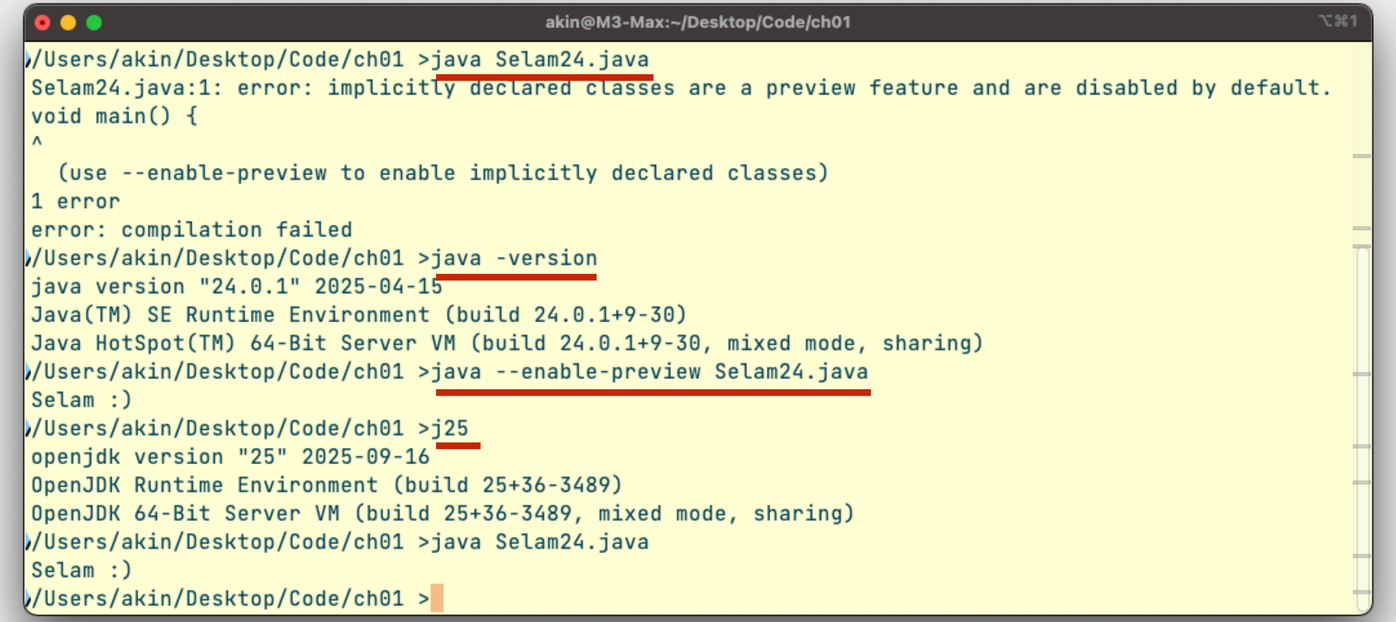


- Her yeni Java sürümünde henüz detayları kesinleşmemiş ama Java topluluğunun takdirine sunulan özellikler olur
- Bunlara *gözden geçirme* (*preview*) özellikleri denir
- Java topluluğunu önerileri doğrultusunda gerekirse özellikte değişiklikler yapılabilir hatta tamamen iptal edilebilir
- Gözden geçirme (preview) özellikleri henüz standart hale gelmediğinden ancak **`--enable-preview`** seçeneği (ya da parametresiyle) (flag) çalıştırılabilir

Gözden Geçirme Özellikleri - II



- Örneğin, ben Java 24 ile **`Selam24.java`**'yı çalıştırmak istediğimde, bu kodda henüz *gözden geçirme* (*preview*) safhasında özellik olduğu gerekçesiyle hata aldım ve ancak **`java --enable-preview Selam24.java`** komutuyla çalıştırabildim

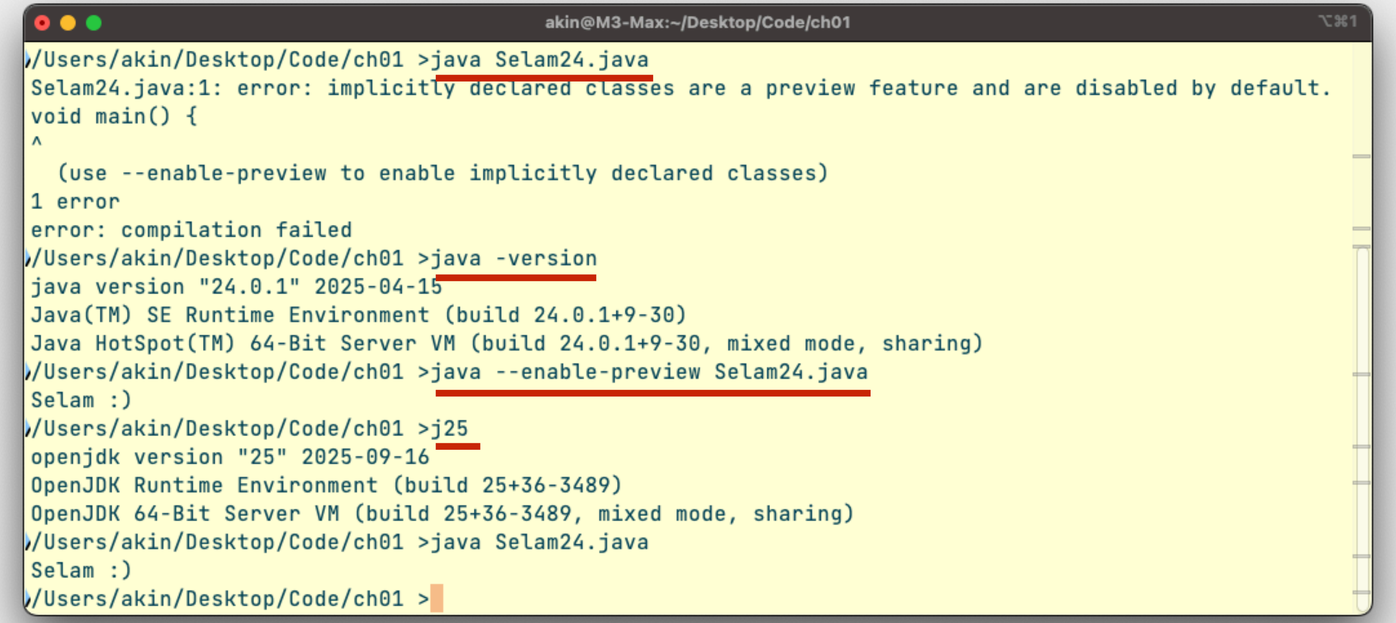


Gözden Geçirme Özellikleri - III



- Ama Java sürümünü 25'e çıkardığımda artık kesinleşen bu özelliği `--enable-preview` olmadan kullanabildim

- Benzer şekilde 26 ve 27. sürümlerle de bu kod çalışacaktır



Bölümün Sonunda



- Bu ilk bölüm zaten Java geliştirme ortamını kurmak için, dolayısıyla birazdan gerekli kurulumları yapıp ayarları tamamladığınızda bu kodları çalıştırabilir hale geleceksiniz

Soru ve Cevap Zamanı!





Makinemde Java Var mı?



Makinemde Java Var mı? - I



- Bir makinede Java olup olmadığı nasıl anlaşılır?
- Bir makinede Java olması demek o makinede ya JDK ya JRE ya da ikisinin birlikte kurulu olması demektir
  - Unutmayın, geliştirmek için JDK çalıştırmak için JRE olmalı!
  - OpenJDK ve Oracle JDK dağıtımları için Java 11 ile birlikte artık kurulum olarak sadece JDK'nın olduğunu belirtmiştik
  - Ama diğer dağıtımlarda hala JRE olabilir de demiştik

Makinemde Java Var mı? - II

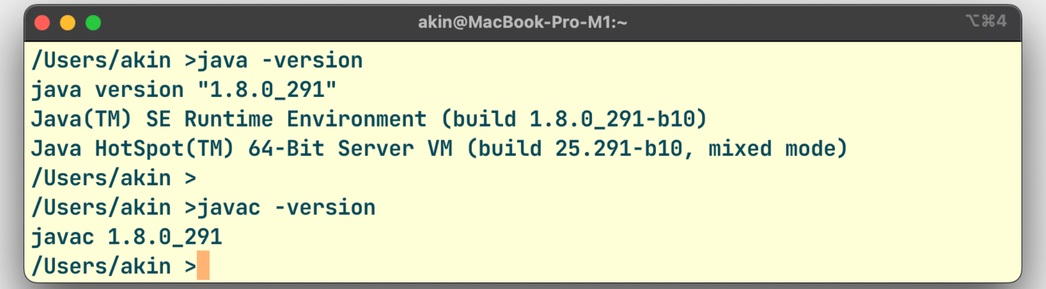


- Bir makinede JDK olup olmadığını anlamanın en hızlı yolu komut satırında **`javac -version`** komutunu çalıştırmaktır
- JRE için ise **`java -version`** komutu çalıştırılır
  - Ama JRE ayrı kurulmuş ya da JDK içinde gelmiş olabilir
- Bu komutlar bulunamazsa, o makinede JDK ya da JRE olmadığını söyleyemeyiz ama olsa bile en azından gerekli ayarların yapılmadığını, dolayısıyla bunlara komut satırından ulaşılamadığını söyleyebiliriz

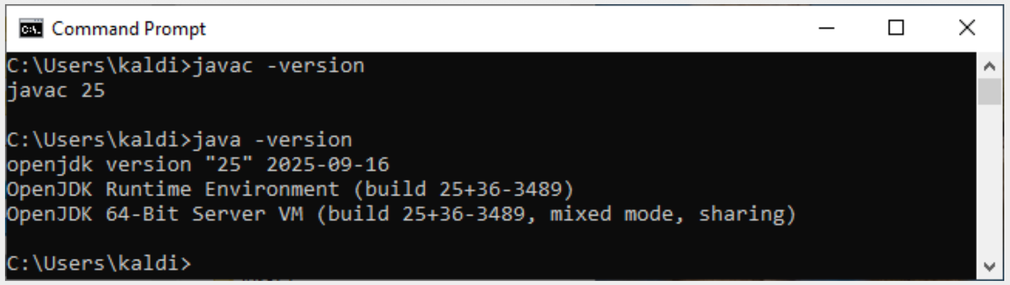
Makinemde Java Var mı? - III



- Burada farklı İS'lerinde komut satırında `javac -version` ve `java -version` komutlarının cevapları görülmektedir









Makinemde Java Var mı? - IV



- Aynı komutları makinanızda komut satırında siz de çalıştırın ve hata almıyorsanız, sonuca göre JDK olarak neyin kurulu olduğunu anlayın
- Daha önce söylemiştik, **`java`** komutunu çalıştırdığınızda bulunamadığı gerekçesiyle hata alırsanız dert etmeyin, bu henüz makinanızda Java geliştirme ortamı olmadığı ya da olsa bile gerekli ayarlar yapılmadığı içindir, bu bölümde kurulumları tamamlayacağız

Birden Fazla JDK ve JRE - I



- Şu durumlara da dikkat etmek gerekir:
  - Bir makinede birden fazla, farklı sürümlerde JDK ve JRE olabilir
    - Kuruluma göre, yeni güncelleme eskisini silebilir, yani JDK'nın 25. sürümü varken 25.0.2 güncellemesi kurulursa 25. sürüm silinebilir
  - Dolayısıyla komut satırından ulaşılan JDK ya da JRE, makinedeki tek kurulum olmayabilir

Birden Fazla JDK ve JRE - II



- IDE'lerde ise durum farklıdır:
  - Genelde IDE'ler **`JAVA_HOME`** değişkeni ile belirtilen JDK'yı kullanır
  - IDE'lerdeki projeler varsayılan durumda IDE'nin kullandığına ulaşır ama projelere has tanımlar da yapılabilir
    - Örneğin birden fazla JDK ya da JRE'nin tanıtıldığı durumlarda IDE'nin bir projede hangisini kullanacağının ayarı yapılabilir
- Tüm bu ayarları birazdan ele alacağız

JDK ve JRE Nerededir? - I



- Eğer makinemde JDK(ler) ya da JRE(ler) varsa, nerededirler?
- Bu sorunun cevabı için şunlar göz önüne alınmalıdır:
  - JDK ve JRE'nin, varsayılan yani hiçbir değişiklik yapılmadan kurulduğunda, kurulum yeri, İS'ne ve kurulan JDK (JRE) dağıtımına bağlıdır, hatta aynı işletim sisteminin farklı sürümlerine göre de değişebilir
  - Ayrıca varsayılan yer kullanılmayıp farklı bir yere kurulum yapılmış olabilir

JDK ve JRE Nerededir? - II



- Hatta bazı programlar, eskiden tarayıcılar (browsers) ve Java ile çalışan bazı masaüstü uygulamaları kendi JDK'larını (aslında JRE'lerini, çünkü IDE'ler dışında uygulamaların amacı olsa olsa Java'yı çalıştırmaktır) kurabilirler
- Dolayısıyla, bir makinede hangi JDK ve JRE'lerin olduğunu bilmek için detaylı araştırma yapmak gerekir
  - Bu amaçla İS'nde **`which`**, **`where`**, **`find`** vb. komutlar kullanılabilir

Soru ve Cevap Zamanı!





Java Dağıtımları



Hatırlatma - I



- En baştan şunu söyleyelim: Makinenizde bir JDK 25 kurulumu (installation) varsa ve az önce anlattığım gibi komut satırında **`javac -version`** ve **`java -version`** komutları çalışıyor ve 25. ya da daha yeni 26. veya 27. sürümü gösteriyorsa bile yine de bu bölümü atlamayın
- Siz bu eğitime ulaştığınızda muhtemelen Java'nın 28. hatta daha yeni sürümleri de çıkmış olacaktır, bunlara da makinanızda sahipseniz yine de bu bölümü atlamayın

Hatırlatma - II



- Bu bölümün devamını izlerseniz özellikle iş dünyasında sık karşılaşacağınız durumlarla ilgili şimdiden bilgi sahibi olursunuz
- Örneğin
  - yaygın JDK dağıtımlarını (distributions)
  - herhangi bir ortama nasıl Java kurup ayarlarını yapacağınızı
  - bir makinada birden çok Java kurulumunu nasıl yöneteceğinizi
- vs. öğrenebilirsiniz

Hatırlatma - III



- Ama bu bölüm size sıkıcı gelirse 😀 ve halihazırda makinanızda Java 25 veya daha yeni bir sürüm varsa doğrudan 2. bölüme geçip, bu bölüme daha sonra, ara ara uğrayarak eksiklerinizi giderebilirsiniz



OpenJDK

OpenJDK - I



- JDK (ya da JRE) kurulumları (installation) için en temel yer **OpenJDK**'dır:
  - https://openjdk.org/
- OpenJDK, Java SE'nin açık kaynak kodlu (open source) gerçekleştirmesidir ve 2006'da Sun'ın bağışladığı kod tabanıyla (code base) oluşturulmuştur
- Bazı istisnalar dışında bütün JDK dağıtımları OpenJDK üzerine kuruludur

OpenJDK - II



- OpenJDK'da hem geçmiş hem gelecek sürümlere, JDK'ların getirdiği yeniliklere, projelerine, ilerlemelerine vs. ulaşılabilir
  - JDK 25: https://openjdk.org/projects/jdk/25/
- OpenJDK aslen bir kaynak kod (source code) geliştirme projesidir
- Dolayısıyla herhangi bir dağıtımı (binary distribution) yoktur, kurumlar OpenJDK'yı kullanarak kendi dağıtımlarını sağlar

OpenJDK JVM



- OpenJDK tabii olarak bir JVM ile gelir
- Bu JVM'in adı **HotSpot**'tır ve çoğunlukla C++ ile geliştirilmiştir
- HotSpot dışında başka JVMler de geliştirilmektedir:
  - Oracle GraalVM
  - Eclipse OpenJ9
  - Azul Zing, SAP JVM, Alibaba Dragonwell ve diğerleri

java.net



- OpenJDK'nın dağıtımlarından birini Oracle yapar: https://jdk.java.net/
- MS Windows, macOS ve Linux paketleri için bu OpenJDK dağıtımları kullanılabilir
  - JDK 25: https://jdk.java.net/25/
- OpenJDK dağıtımları tamamen ücretsizdir; buradan indirip makinanıza kurabilirsiniz



Oracle JDK

Oracle'ın JDK Dağıtımı - I



- Oracle, Java'nın, MS Windows, macOS, Linux için, **Oracle JDK** isimli başka bir dağıtımını da sağlar: https://www.oracle.com/java/technologies/downloads/
- Bu da OpenJDK üzerine kurulmuştur, aynı koda sahiptir
- Oracle kendi JDK dağıtımına bazen deneysel (experimental) yapıları dahil eder
- Dolayısıyla Oracle JDK ve OpenJDK temelde aynıdır

Oracle'ın JDK Dağıtımı - II



- Oracle JDK, 11. sürüm öncesinde tamamen bedavaydı
- Oracle 11 ve 16 arasındaki sürümler için JDK'sının canlı ortamda kullanımı (production use) için ücretli lisans istedi, geliştirme/test için bedava tuttu
  - Bu sırada OpenJDK'ya büyük geçiş oldu
- Oracle, 17. sürümle birlikte her türlü kullanımı, bir sonraki LTS'in çıkışından itibaren 1 sene sonrasına kadar ücretsiz yapmıştır, sonrasında lisans ücreti ödenmelidir ya da yeni LTS'e geçilmelidir

Oracle'ın JDK Dağıtımı - III



- Örneğin dersimizde kullandığımız JDK 25, bir sonraki LTS olan 29. sürüm Eylül 2027'de çıktıktan 1 sene sonrasına, yani Eylül 2028'e kadar ücretsiz güncellenir
- Bu süre dolunca sonraki güncellemeler ücretli olur

Oracle'ın JDK Dağıtımı - IV



- Bahsedilen Oracle JDK'nın ücretli-bedava olması sadece canlı ortam kullanımı (production use) içindir
  - Öğrenme, deneme, geliştirme, test vs. için tamamen bedavadır
- Ben genelde Oracle JDK kullanıyorum, bu derste de Oracle JDK'nın 25. sürümünü kullanacağım



Diğer Dağıtımlar

Diğer Dağıtımlar - I



- Ayrıca pek çok başka ücretsiz JDK dağıtımı da vardır:
  - Microsoft Build of OpenJDK https://www.microsoft.com/openjdk
  - Eclipse Temurin https://adoptium.net/temurin
  - Amazon Corretto https://aws.amazon.com/corretto/
  - Ve diğerleri

Diğer Dağıtımlar - II



- Tüm bu dağıtımlar da OpenJDK üzerine kurulmuştur ve içerdikleri fazladan araçlara, uyum özelliklerine vs. sahip olabilirler
- Örneğin Microsoft Build of OpenJDK, Azure bulut servisleri (cloud services) için desteğe sahipken Amazon Corretto, AWS desteğine sahiptir



Hangisini Kurayım?

Hangisini Kurayım? - I



- Makinanıza ister Oracle'ın JDK'sını isterseniz Oracle'ın OpenJDK'sını isterseniz de farklı bir JDK, mesela Microsoft'un JDK'sını veya Eclipse Temurin JDK'sını kurun, dersimiz açısından değişen hiçbir şey olmaz
- Çünkü Java standarttır ve platformdan bağımsızdır dolayısıyla örneğin derleyicinin nerede hata vereceği ya da çalışma zamanının nasıl başlatılacağı vb. detaylar, kütüphanesinin kullanımı vs. her kurulumda aynıdır
- Zaten tüm bu dağıtımların OpenJDK tabanlı olduğunu da söylemiştik

Hangisini Kurayım? - II



- Ama ben size birazdan, MS Windows, macOS ve Linux platformları için kurulum tavsiyesi vereceğim, onlara uyarsanız rahat edersiniz

Soru ve Cevap Zamanı!





Java Ayarları



Java Ayarları - I



- Kurulumlara geçmeden önce kısaca iki ayardan bahsedelim
- Bir makinaya bir JDK (ya da JRE) kurulduğunda ona ulaşabilmek için **`PATH`** ve **`JAVA_HOME`** sistem değişkenlerinin ayarlanması gerekir
- **`PATH`**: İS'lerinde çok yaygın olan ve çalıştırılabilir (executable, exe) tüm komut ve programların patikalarını (path) gösteren bir sistem değişkenidir, bir ya da daha çok patikaya sahiptir
- **`JAVA_HOME`**: Java'ya has bir sistem değişkenidir, tek bir değeri olabilir ve bu da kurulmuş olan JDK'nın dizinidir, JDK için *PATH*'tir denebilir

Java Ayarları - II



- Programlar Java'ya ulaşmak için bu iki değişkeni kullanırlar, dolayısıyla bu ayarlar yapılmazsa makinanızda JDK kurulu olsa bile Java kullanmak isteyen programlar ona ulaşamazlar

PATH - I

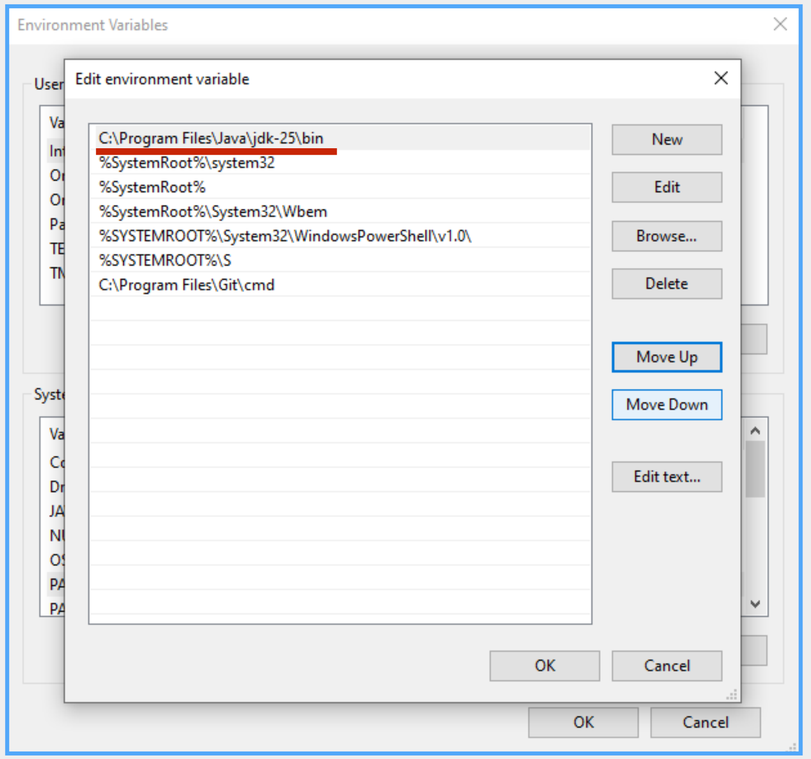


- **`PATH`**, *İS'lerinde çalıştırılabilir tüm komut ve programların patikalarını gösteren bir sistem değişkenidir* dedik
- İS'leri bir programı çalıştırmak istediklerinde bu programın çalıştırılabilir dosyasını tüm diskte aramak yerine sadece **`PATH`** değişkeninde listelenen dizinlerde arar
- Eğer çalıştırılabilir bir dosyanın dizini **`PATH`** değişkeninde listelenmiyorsa bu durumda o dosya çalıştırılamaz

PATH - II



- Bazı programlar da Java'ya erişmek için **`PATH`**'i kullanır, dolayısıyla bu değişken JDK'nın **`bin`** dizinini içerecek şekilde ayarlanmalıdır
  - Örneğin Windows bir makinaya Oracle JDK 25 kurulursa, **`bin`** dizini **`C:\Program Files\Java\jdk-25\bin`** olur, dolayısıyla da **`PATH`** bu değeri içermelidir

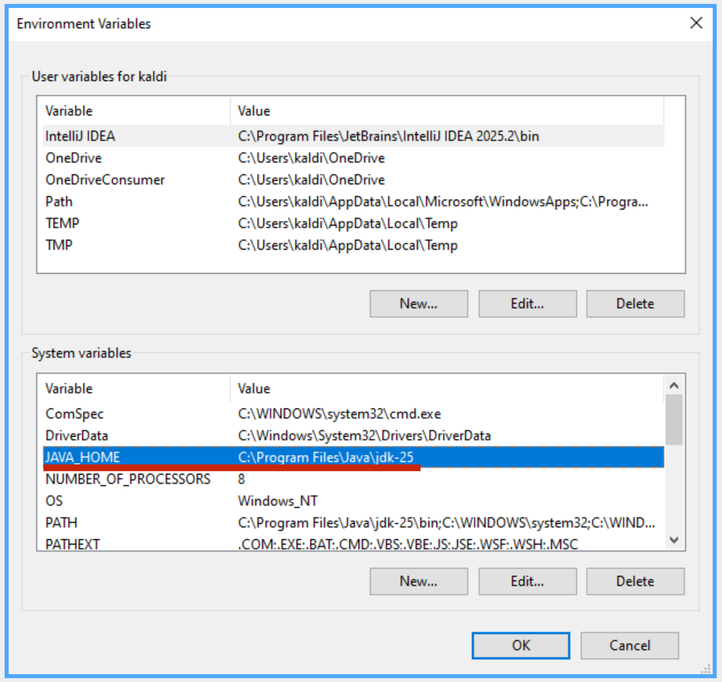


- Dikkat ederseniz bu Windows makinanın **`PATH`**'inde **`%SystemRoot%`** yani **`C:\Windows`** gibi diğer patikalar da görülmektedir

JAVA_HOME



- IntelliJ, Eclipse ve VS Code IDE'leri veya Tomcat gibi programlar, Java'ya erişmek için **`JAVA_HOME`**'u kullanır, dolayısıyla bu değişken JDK'nın dizinini gösterecek şekilde ayarlanmalıdır
  - Örneğin Windows bir makinaya Oracle JDK 25 kurulursa, dizini **`C:\Program Files\Java\jdk-25`** olur, dolayısıyla da **`JAVA_HOME`** değişkeninin değeri bu patika olmalıdır



Java Ayarları - III



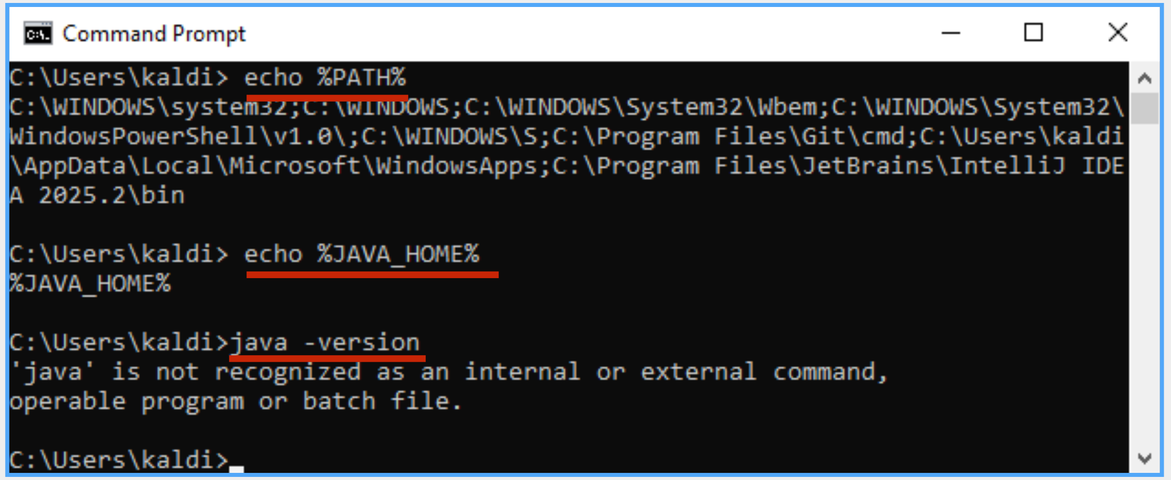
- Bu ayarlara komut satırından da ulaşılabilir
- Benim makinamda bu ayarlar şöyle görünüyor ve hem JDK 25'in **`bin`** dizini **`PATH`**'te hem de **`JAVA_HOME`** ayarlanmış durumda



Java Ayarları - IV



- Ama Windows makinamda **`PATH`**'te herhangi bir JDK olmadığı gibi **`JAVA_HOME`** da ayarlanmamış halde dolayısıyla **`java -version`** dediğimde **`java`** komutunu bulamıyor



Java Ayarları - V

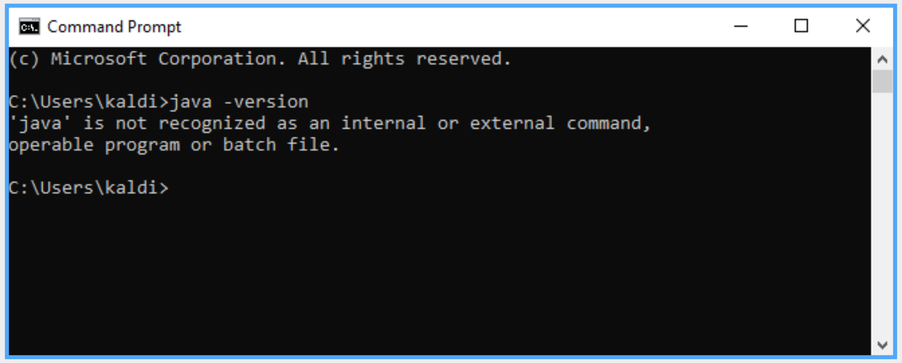


- Makinanızdaki programlardan Java'yı kullananların hangisinin **`PATH`**'i hangisinin **`JAVA_HOME`**'u kullandığını bilemeyebilirsiniz, dolayısıyla bu iki ayar da tutarlı bir şekilde yapılmalıdır, aksi taktirde **`javac`** ve **`java`** komutlarına ulaşılamaz ve hata alırsınız

Java Ayarları - VI



- Örneğin Windows 10'un komut satırında **`java -version`** girildiğinde İS önce bulunulan dizinde sonra da **`PATH`** değişkenindeki dizinlerde bu komutu arar, bulduğunda çalıştırır, bulamazsa hata verir



Soru ve Cevap Zamanı!





JDK Kurulumu



Kurulum - I



- Yazılım kurulumlarının yaygın iki şekli vardır:
  - İndirilen kurulum dosyasının "next/continue" vb. tıklamalarla sistem tarafından otomatik olarak kurulması:
    - Bu, MS Windows ve macOS için en yaygın durumdur
      - Kurulum sırasında yer vb. seçenekler sorulabilir
    - Linux Ubuntu'da ise paket yöneticileri (package manager) ile kurulumlar yapılır, örneğin komut satırında **`apt`** ya da **`apt-get`** komutlarıyla bir listedeki yazılımlar kurulur (install) ya da güncellenir (update)

Kurulum - II



- İkinci yöntem ise indirilen sıkıştırılmış (zip, tar, gzip vs.) dosyanın istenilen yere açılmasıdır
  - Bu durumda ilk yöntemde otomatik yapılan bazı ayarların elle yapılması söz konusu olabilir

JDK (JRE) Kurulumu



- JDK (eskiden JRE) kurmak için:
  - İlk yöntemde, platforma özel JDK kurulum dosyası (Windows için **`exe`** ya da **`msi`**, macOS için **`dmg`** veya **`pkg`**) indirilir ve *next/continue* vb. tıklamalarla kurulur
  - Linux'ta kurucu paket olarak, dağıtıma göre RPM veya Debian (.deb) paketi indirilebilir
  - İkinci yöntemse 3 İS için de geçerlidir, sıkıştırılmış, **`zip`**, **`gzip`** ya da **`tar`** dosya indirilir ve açılarak istenilen yere konur

Oracle JDK Kurulumu - I



- Tüm Oracle JDK kurulumları için: https://www.oracle.com/java/technologies/downloads/
- MS Windows ve macOS için en kolay kurulum kurucu (installer) ile yapılır
- Linux'ta en kolay kurulum dağıtıma göre değişir: RHEL, Fedora, SUSE gibi RPM tabanlı dağıtımlarda RPM, Debian ve Ubuntu'da ise Debian (.deb) kurucu paketi indirilir

Oracle JDK Kurulumu - II



- Oracle JDK kurulduğunda `PATH` ayarının yapılıp yapılmayacağı platforma göre değişir ama `JAVA_HOME` ayarı yapılmaz, elle (manual) yapmak gerekir, bundan dolayı birazdan vereceğim kurulum yöntemlerini uygulayalım
- Oracle ayrıca JDK kurulumları için **JDK Installation Guide** sağlamaktadır
- JDK 25 için: https://docs.oracle.com/en/java/javase/25/install/overview-jdk-installation.html
- Her yeni sürüm çıkınca kurulum dokümanına bakabilirsiniz

Platform Kurulumları



- Hangi JDK dağıtımını veya sürümünü kurarsanız kurun, burada anlattıklarım geçerlidir; örneğin Mart 2027'de Java 28, sonrasında daha yeni sürümler, çıkınca benzer şekilde kurabilirsiniz
- Şimdi MS Windows, macOS ve Linux için tavsiye edeceğim kurulumlarını görelim

Soru ve Cevap Zamanı!





MS Windows'ta JDK Kurulumu



MS Windows'ta JDK Kurulumu - I



- Windows'ta JDK kurmanın üç yolu vardır ve **`PATH`** ile **`JAVA_HOME`** ayarları kuruluma göre değişir:
  - Eclipse Temurin ve Microsoft Build of OpenJDK: kurucu **`PATH`**'i ayarlar, isterseniz **`JAVA_HOME`**'u da
  - Oracle JDK: kurucu **`java`** ve **`javac`**'ı **`PATH`**'e ekler, **`JAVA_HOME`**'u sizin ayarlamanız gerekir
- Sıkıştırılmış (zip) kurulumlar: **`PATH`** ve **`JAVA_HOME`**'u tamamen siz ayarlarsınız

MS Windows'ta JDK Kurulumu - II



- Önce Oracle JDK'yı kuracağız; kurucusu `PATH`'i ayarlar ama `JAVA_HOME`'u ayarlamaz
- `JAVA_HOME`'u elle tanımlayıp `PATH`'i de ona bağlayacağız; zip kurulumlarda yapmanız gereken de tam olarak budur
- Sonra bu ayarların hepsini sizin için yapan Temurin ve Microsoft Build of OpenJDK kurulumlarını göreceğiz



MS Windows'ta Oracle JDK Kurulumu

Oracle JDK - I



- Oracle JDK, https://www.oracle.com/java/technologies/downloads/ adresinden x64 Installer ya da x64 MSI Installer olarak indirilir ve *next/continue* ile, varsayılan ayarlarla kurulur
- JDK 25 şu dizine kurulur: **`C:\Program Files\Java\jdk-25`**
- Kurucu ayrıca **`C:\Program Files\Common Files\Oracle\Java\javapath`** dizinini **`PATH`**'e ekler; **`java`**, **`javac`** ve **`jshell`** komutları bu sayede hemen çalışır
- **`JAVA_HOME`** ise ayarlanmaz; onu birazdan elle ayarlayacağız

Oracle JDK - II



- 64 bitlik platforma 32 bitlik JDK/JRE kurulumları aşağıdaki dizine yapılır:

```
C:\Program Files (x86)\Java\
```

- 32 bitlik JDK/JRE kurulumuna olsa olsa çalıştığınız yerde olan çok eski bir sistem için ihtiyacınız olur

Soru ve Cevap Zamanı!





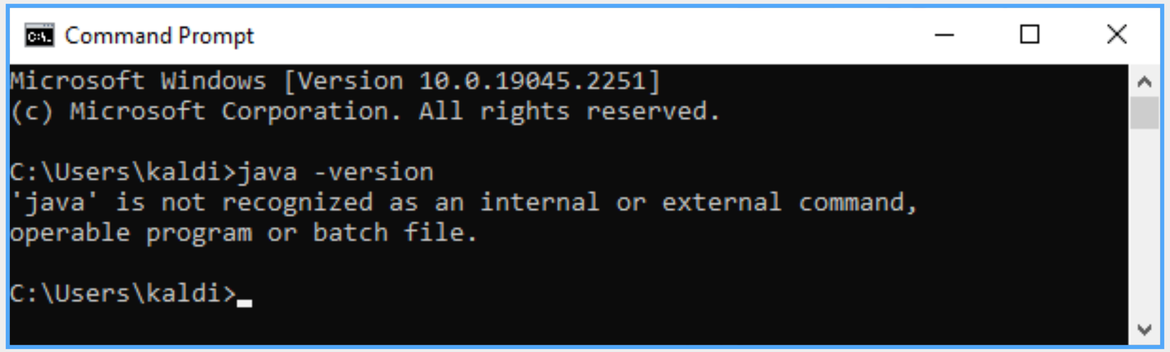


MS Windows'ta PATH ve JAVA_HOME Ayarları

PATH ve JAVA_HOME



- JDK'yı sıkıştırılmış (zip) bir dosyadan kurduysanız, **`PATH`**'i siz ayarlamadıkça komut satırı **`java`** ve **`javac`**'ı bulamaz, aşağıdaki hata bundandır:



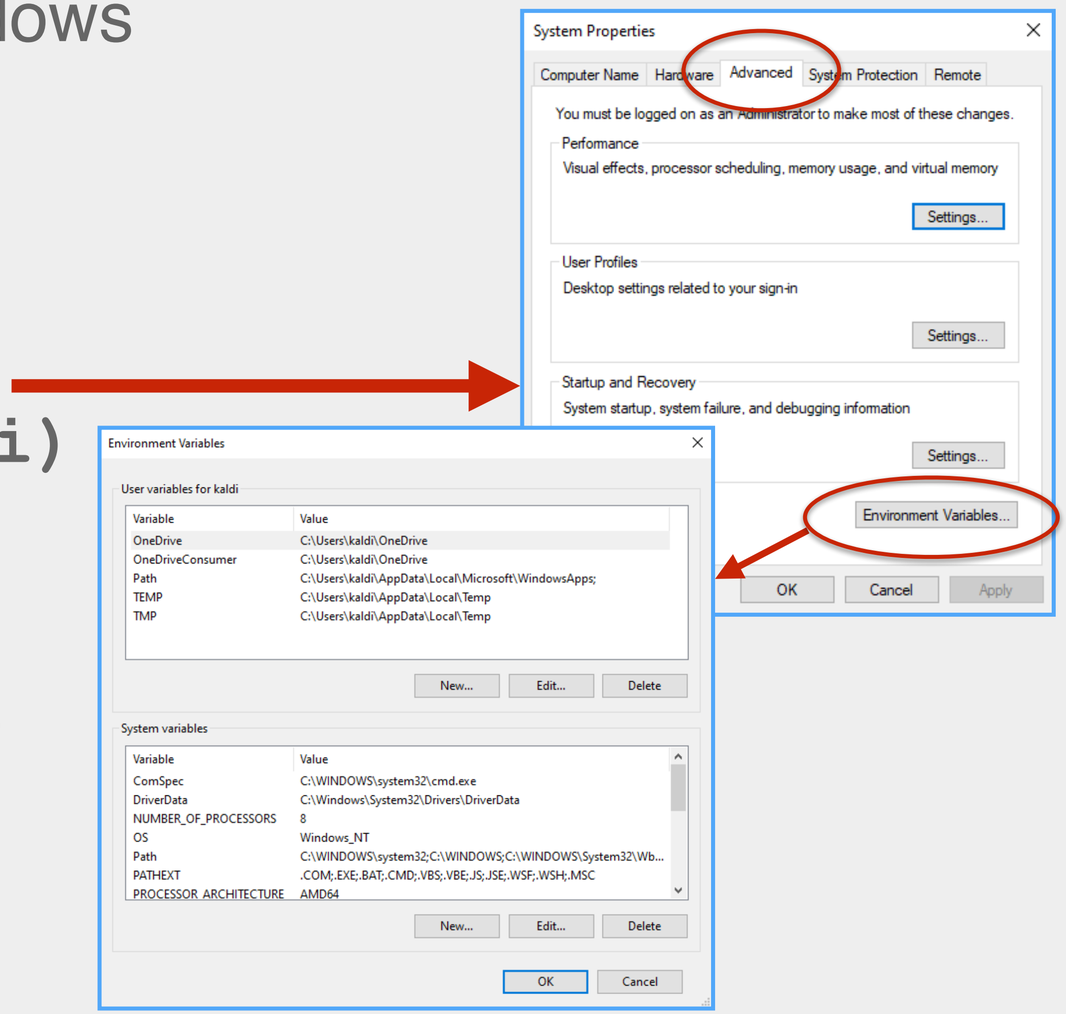
- Oracle JDK'da **`PATH`** kurucu tarafından ayarlanır ama **`JAVA_HOME`**'u yine sizin tanımlamanız gerekir; IDE'ler ve pek çok araç Java'yı **`JAVA_HOME`**'dan bulur

- Dolayısıyla şimdi bu ayarların nasıl yapılacağını görelim

PATH Ayarı - I



- **`PATH`** ayarını yapmak amacıyla Windows makinalarda **`Advanced System Settings`**'e (**`Gelişmiş Sistem Ayarları`**) gidilip **`Environment Variables`** (**`Ortam Değişkenleri`**) düğmesi tıklanır

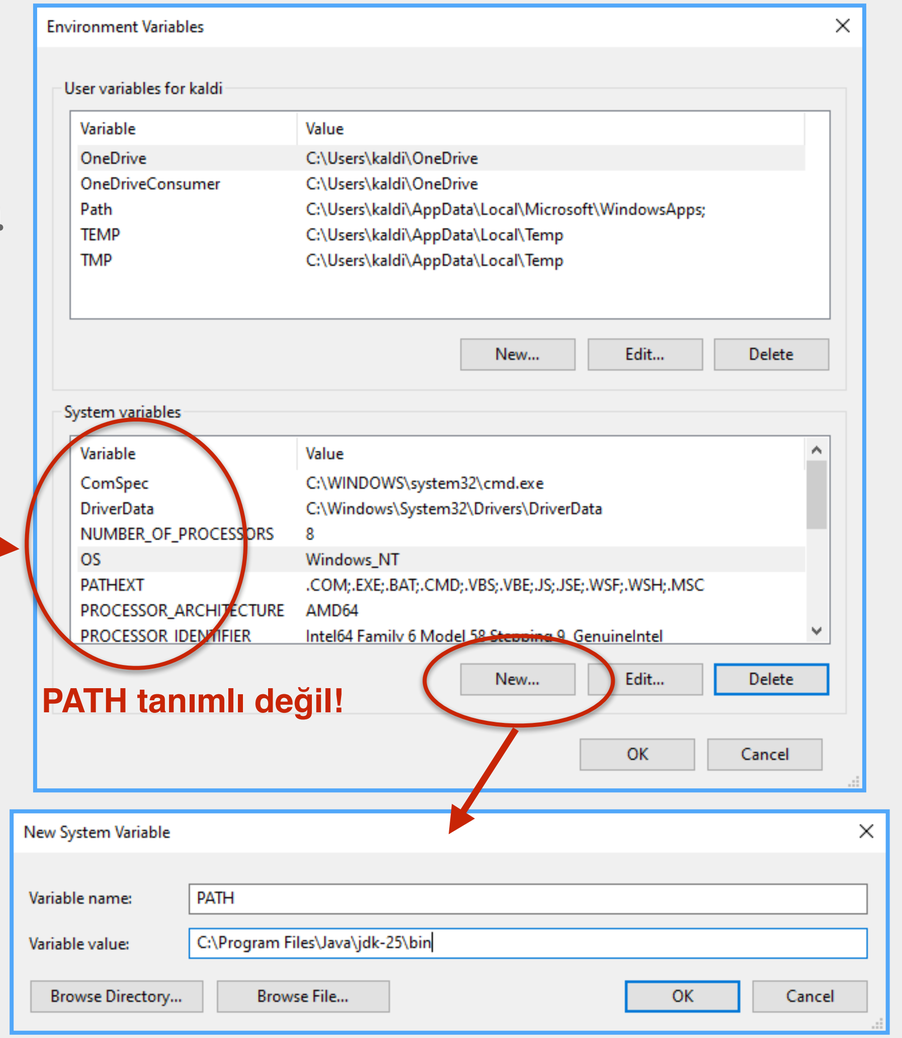


- **`PATH`** ve **`JAVA_HOME`** ayarları burada yapılabilir, biz de bu sırayla ilerleyeceğiz

PATH Ayarı - II



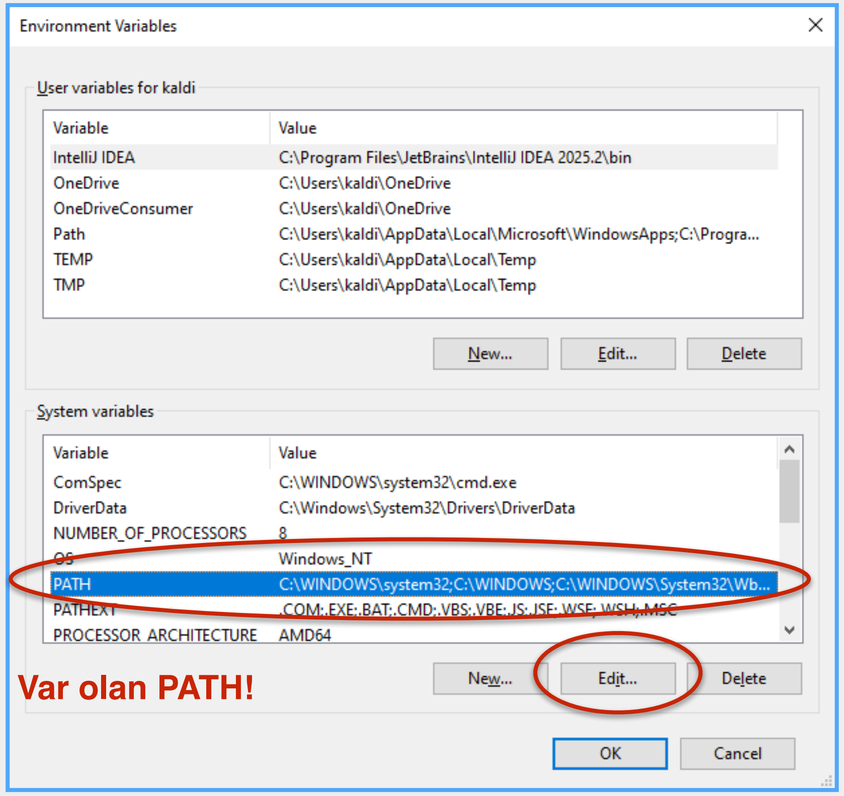
- **`System Variables`**'da muhtemelen **`PATH`** değişkeni tanımlıdır; bu durumda JDK'nın **`bin`** dizininin patikası değişkene eklenir
- Eğer **`PATH`** değişkeni tanımlı değilse, **`New`**'ya tıklayarak açılan pencerede **`PATH`** ve kurulan JDK'nın **`bin`** dizininin patikasını, mesela **`C:\Program Files\Java\jdk-25\bin`** adresini girip **`OK`** tıklarsanız **`PATH`** değişkenini bu değerle tanımlamış olursunuz



PATH Ayarı - III



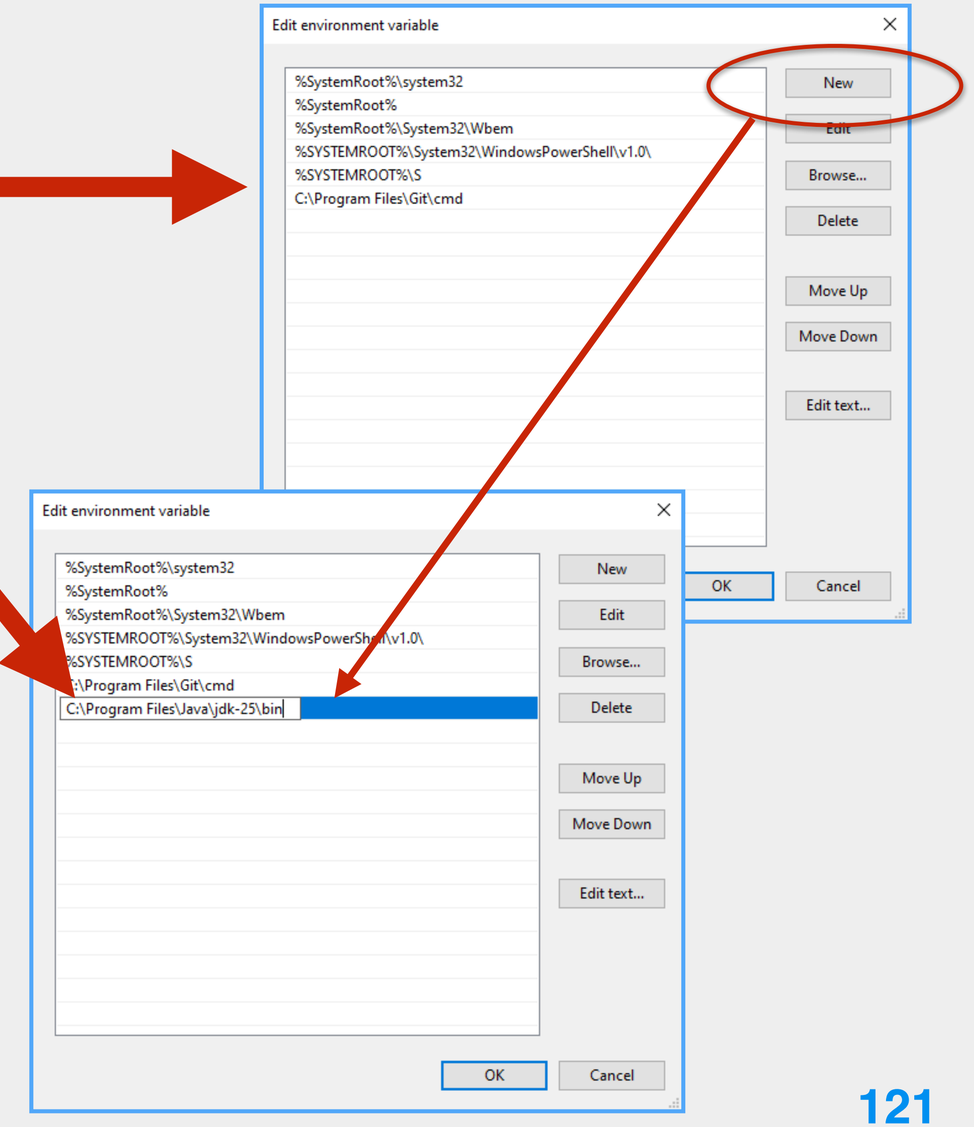
- **`System Variables`** kısmında halihazırda zaten bir **`PATH`** değişkeni varsa bu durumda yapılacak şey, **`C:\Program Files\Java\jdk-25\bin`** adresini **`PATH`**'e eklemektir
  - Unutmayın, **`PATH`** değişkeninde birden fazla patika olur
- Bu amaçla **`PATH`** değişkeni seçilip **`Edit`** düğmesine basılır



PATH Ayarı - IV

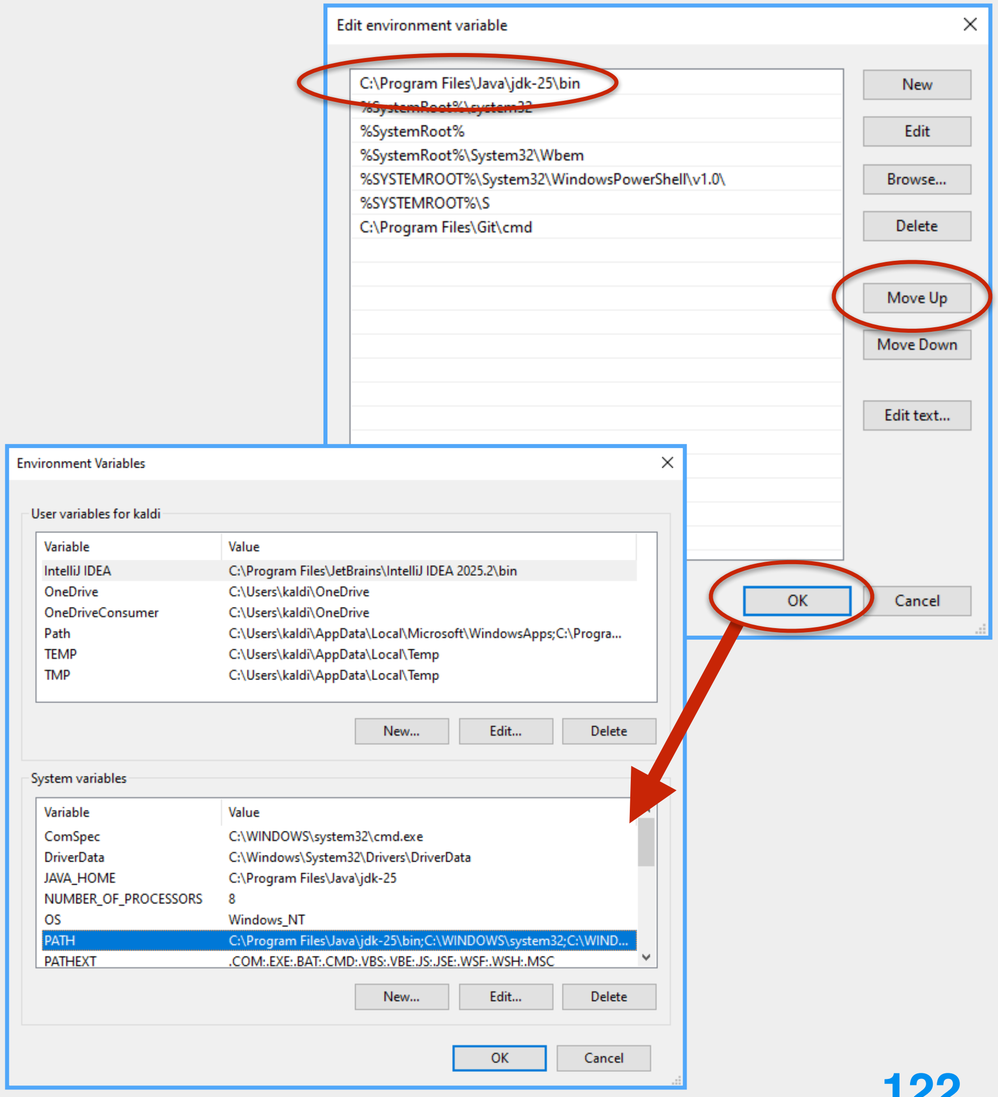


- Açılan pencerede, **`PATH`** değişkeninde bulunan patikaların sıralandığı görülür
- **`New`** düğmesi tıklanarak çıkan pencerede **`C:\Program Files\Java\jdk-25\bin`** adresi değer olarak girilir ve **`OK`**'ye tıklanır
- Bu yeni değer **`PATH`** değişkenindeki patikalar arasında son sırada yerini alır



PATH Ayarı - V

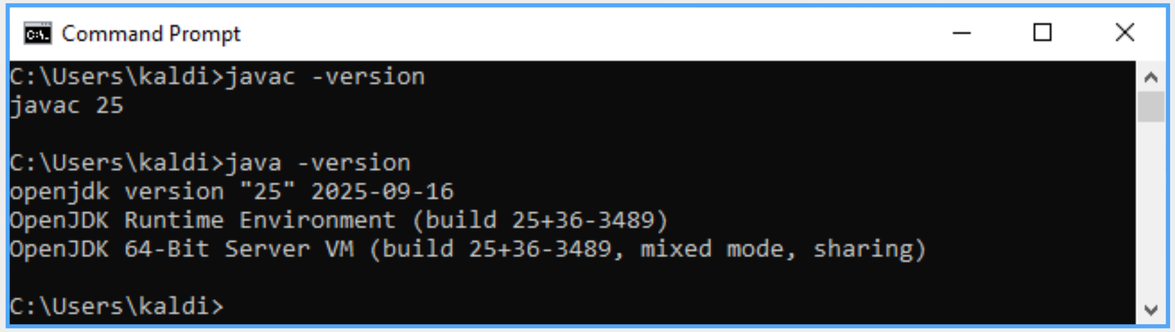
- Girilen patikanın **`PATH`** değişkeninde en başta olmasını sağlamak için **`Move Up`** düğmesi ile yeni değer en üste taşınır
  - **`OK`** tıklanınca **`PATH`** tanımlama biter
- **`PATH`** değişkeninde birden fazla JDK'nın **`bin`** patikası varsa aramada daima en önce gelen bulunacağından, bu şekilde yeni girilen değerin daha önce bulunması sağlanır



PATH Ayarından Sonra - I



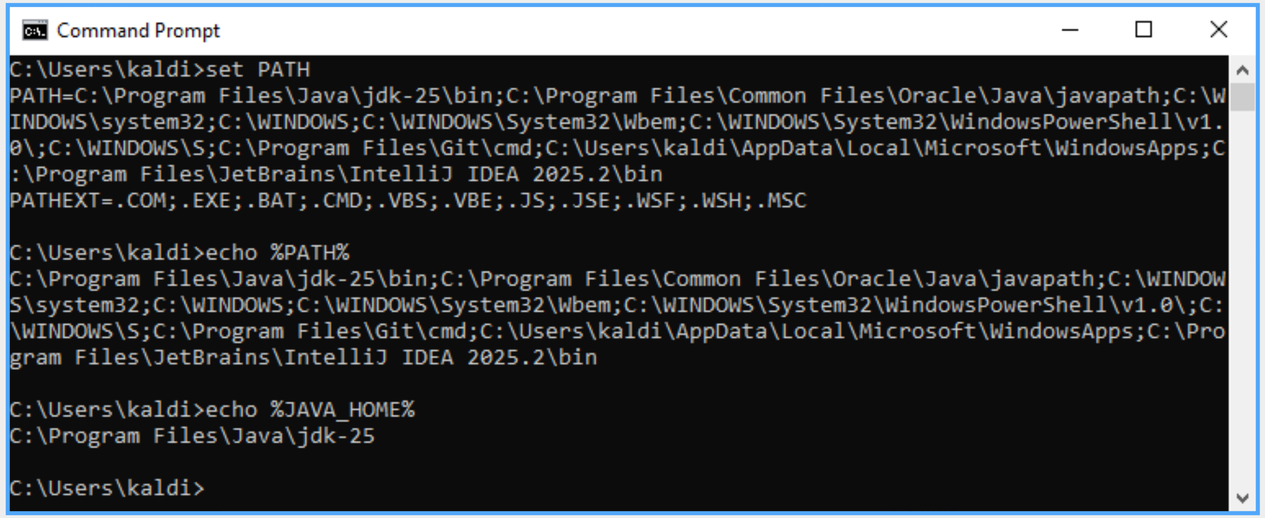
- **`PATH`**'i ayarladıktan sonra açılan komut satırında **`javac -version`** ve **`java -version`** komutları çalışacak ve doğru sonucu verecektir
  - Çünkü komut satırı Java komutları için **`PATH`**'i kullanmaktadır
  - Komut satırı zaten açık ise, değişikliğin etkisini görmek için kapatılıp yenisinin açılması gereklidir
- Örneğin OpenJDK kurduktan sonraki komut sonuçları yanda verilmiştir



PATH Ayarından Sonra - II



- Ayrıca komut satırında **`set PATH`** ya da **`echo %PATH%`** komutları da size **`PATH`** değişkeninin değerini yani bir ya da daha fazla patikadan oluşan listeyi size verecektir
  - Windows'ta patikalar birbirinden `;` ile ayrıldığı için **`PATH`** değişkeninde de patikalar arasında `;` yer alır

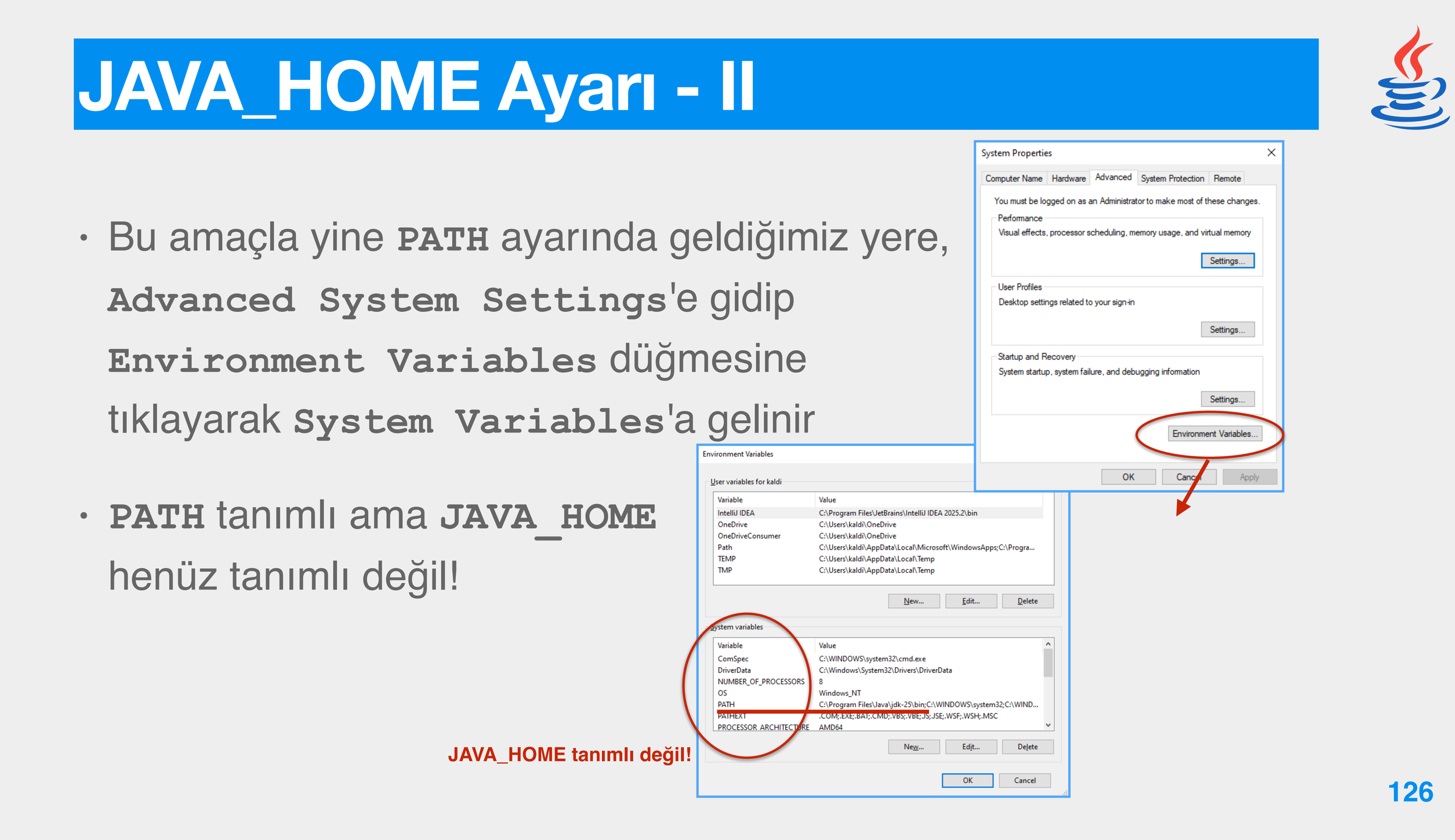


JAVA_HOME Ayarı - I



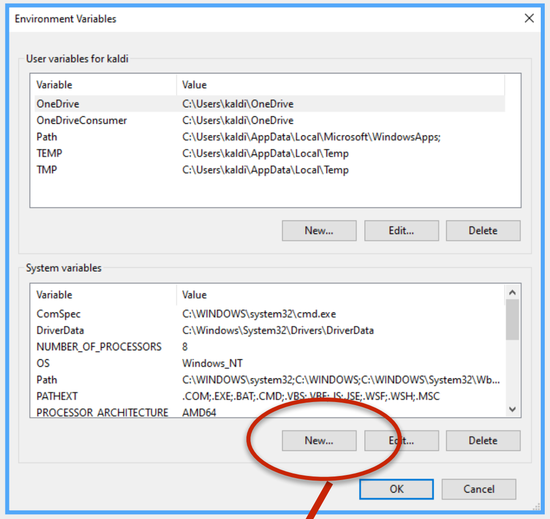
- Bazı programlar, örneğin IntelliJ, VS Code, Java'ya `PATH`'ten değil de `JAVA_HOME` ile ulaşırlar
- `JAVA_HOME`: Java'ya has bir sistem değişkenidir, JDK'nın dizinini gösterir
- Java'yı kullanan programların, Java kurulumuna dolayısıyla da `bin` dizini altındaki çalıştırılabilecek komutlara ulaşabilmeleri için `JAVA_HOME` ile JDK dizininin adresini sisteme tanıtırız

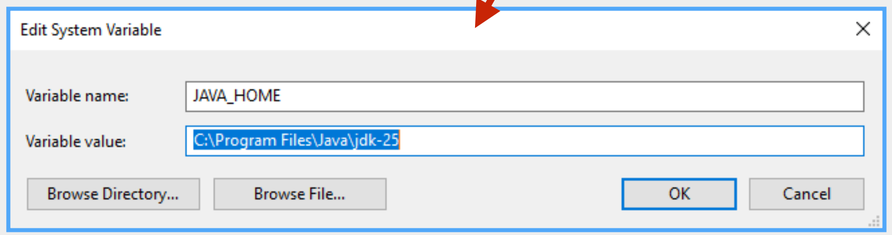
JAVA_HOME Ayarı - II



- Bu amaçla yine **`PATH`** ayarında geldiğimiz yere, **`Advanced System Settings`**'e gidip **`Environment Variables`** düğmesine tıklayarak **`System Variables`**'a gelinir
- **`PATH`** tanımlı ama **`JAVA_HOME`** henüz tanımlı değil!

JAVA_HOME Ayarı - III





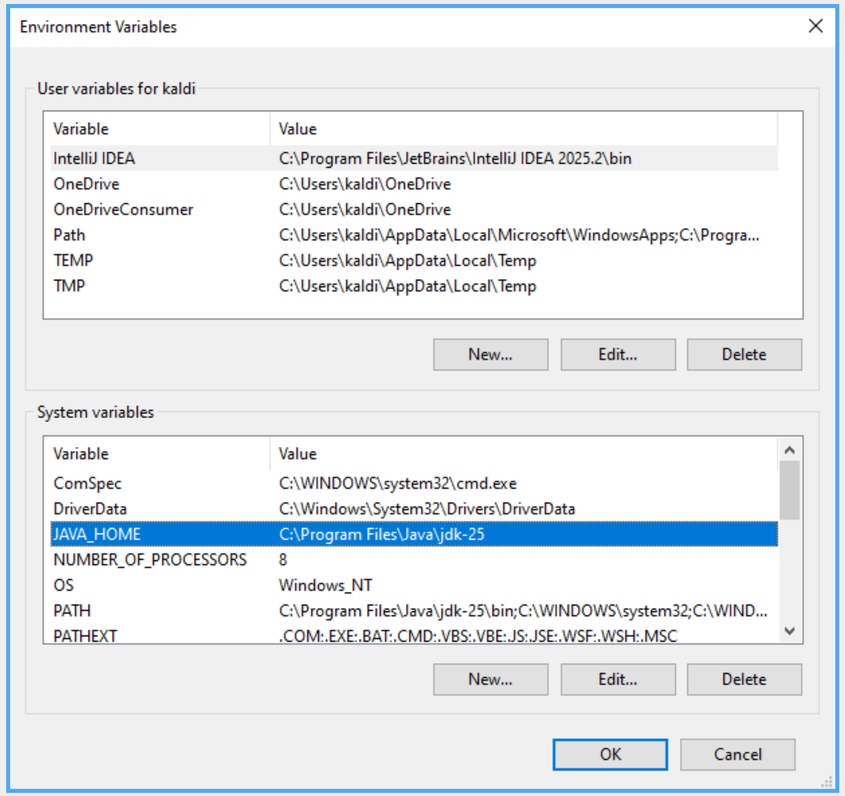
- Sonra **`New`** düğmesine tıklayarak açılan pencerede **`JAVA_HOME`** ve örneğin Oracle JDK kurmuşsanız, onun patikası olan **`C:\Program Files\Java\jdk-25`** adresini girip **`OK`** ile kapatırsanız bu tanımı yapmış olursunuz

- **`JAVA_HOME`**, **`PATH`** gibi değildir, sadece bir tane Java kurulumunu gösterebilir

JAVA_HOME Ayarı - IV



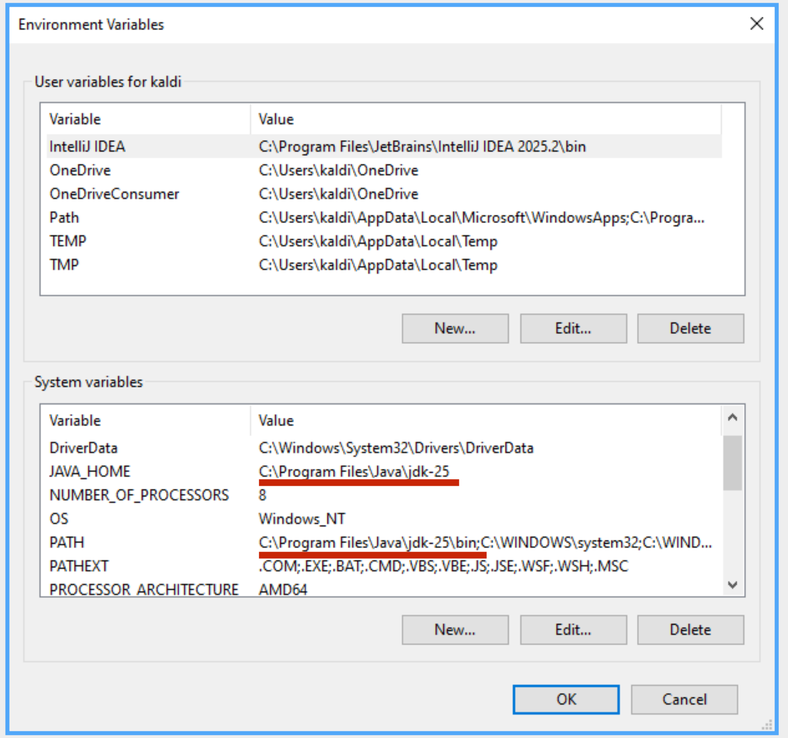
- Bu şekilde tanımlandıktan sonra **`System Variables`**'da **`JAVA_HOME`** görülebilir
- Eğer halihazırda **`JAVA_HOME`** tanımlıysa, seçip, **`Edit`**'e tıklayarak çıkan pencerede değerini değiştirerek yani yeni bir JDK patikası girerek, **`JAVA_HOME`** ile ulaşılan JDK kurulumunu değiştirmiş olursunuz



PATH ve JAVA_HOME - I

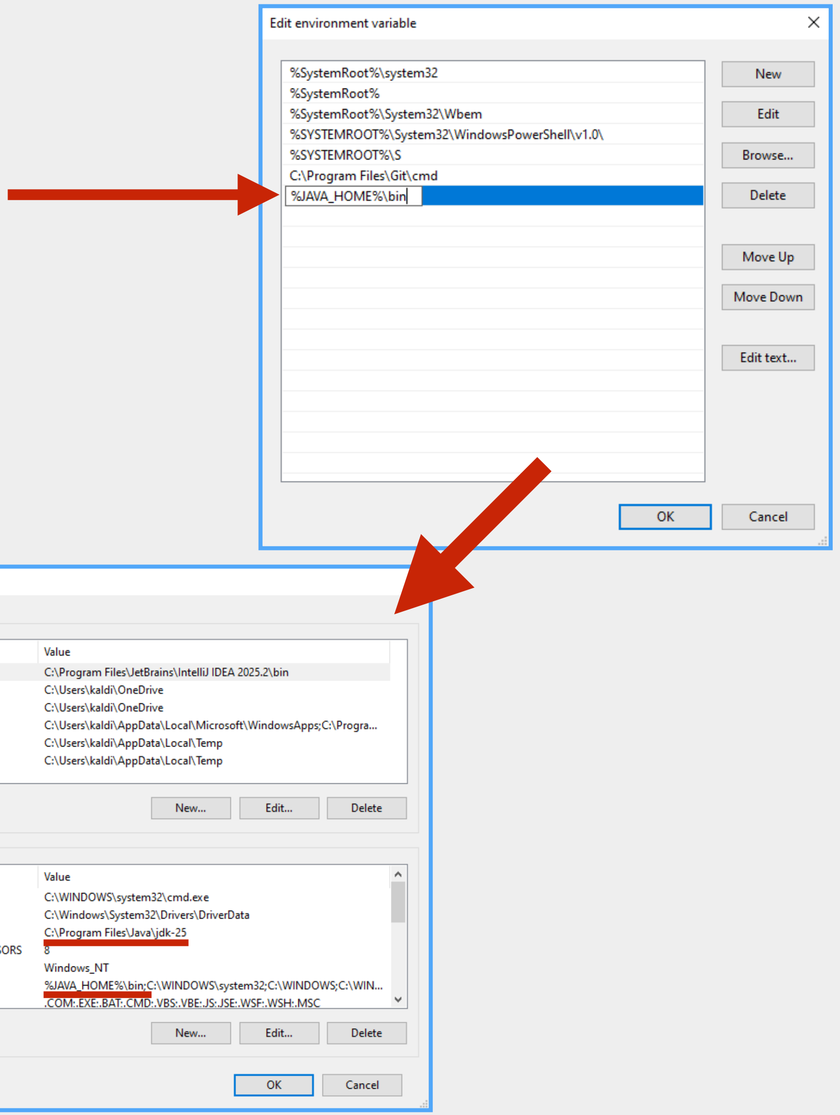


- İS'lerinde **`PATH`** ve **`JAVA_HOME`** ayarlarının uyumlu olması istenebilir
  - Çünkü hem **`PATH`** ile hem de **`JAVA_HOME`** ile aynı Java kurulumuna ulaşmak önemli olabilir
- Bu durumda bu uyuma dikkat etmek için, **`JAVA_HOME`**'da kullanılan kurulumun **`bin`** dizini **`PATH`**'te olmalı hatta birden fazla kurulum olma ihtimaline karşılık ilk sırada yer almalıdır



PATH ve JAVA_HOME - II

- Ya da alternatif ve daha sağlıklı bir yöntem olarak **`PATH`** tanımlarken **`%JAVA_HOME%`** ile **`JAVA_HOME`**'un değerini kullanabilirsiniz
- Girilen patika **`Move Up`** düğmesiyle en üste taşınır ve **`OK`** tıklanır
- Bu durumda **`PATH`** ile erişilen Java **`JAVA_HOME`**'daki Java olur

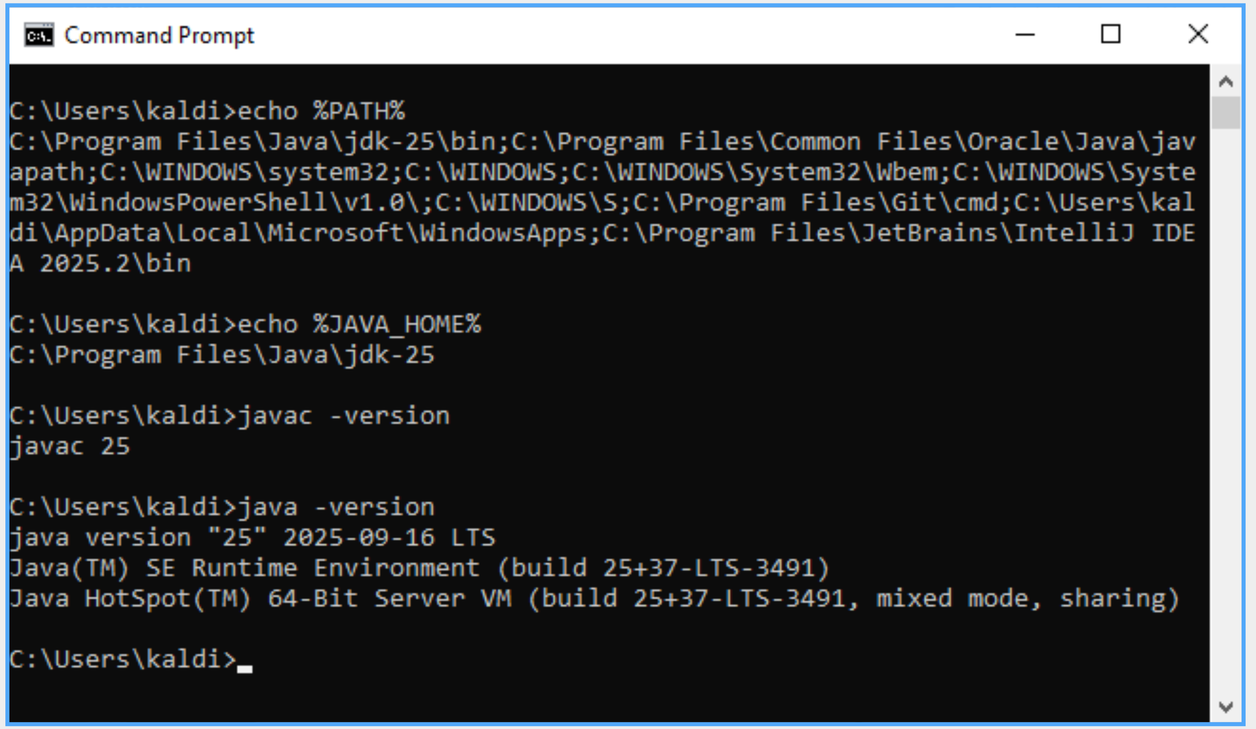


PATH ve JAVA_HOME Ayarından Sonra



- Windows'ta **`PATH`** ve **`JAVA_HOME`** ayarlarından sonraki durum; **`javac`** ve **`java`** komutları çalışıyor ve kurulu sürümü gösteriyor

- Örneğin Oracle JDK kurduktan sonraki komut sonuçları yanda verilmiştir



Soru ve Cevap Zamanı!







MS Windows'ta Temurin ve Microsoft Build of OpenJDK Kurulumu

Temurin ve Microsoft Build of OpenJDK

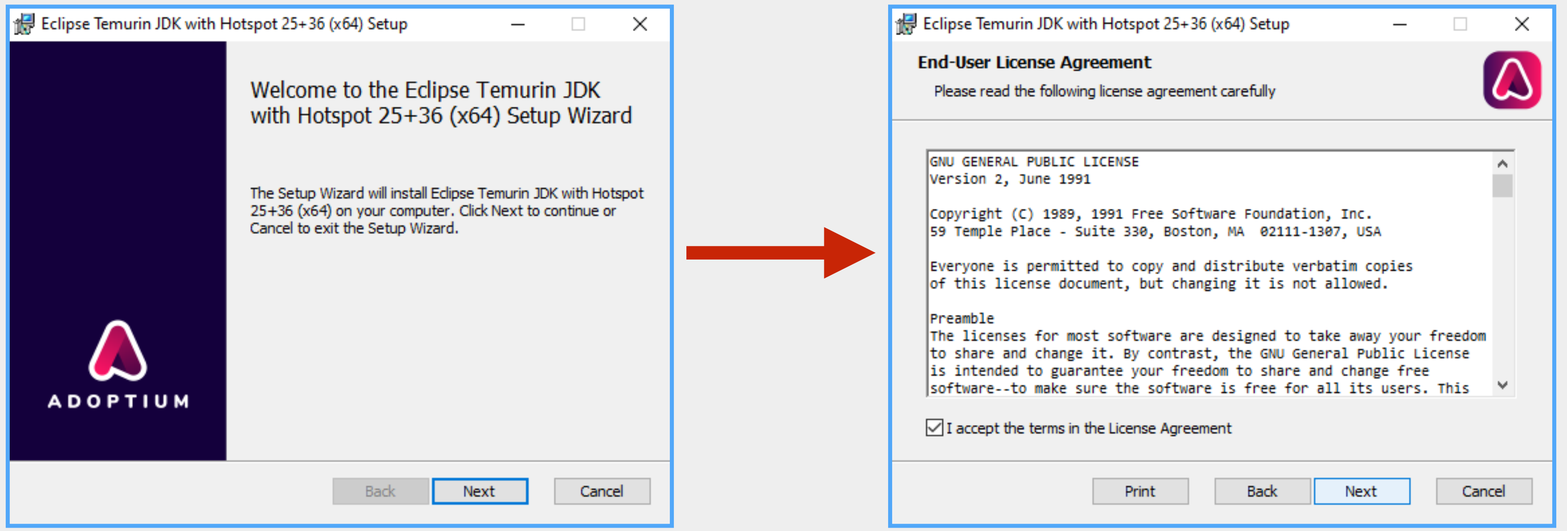


- **`PATH`** ve **`JAVA_HOME`** ayarlarıyla uğraşmak istemeyenler için iki kolay seçenek var:
  - Eclipse Temurin: https://adoptium.net/
  - Microsoft Build of OpenJDK: https://www.microsoft.com/openjdk
- İkisi de OpenJDK'nın paketlenmiş dağıtımıdır ve `.msi` kurucusuyla gelir
- Kurucu **`PATH`**'i kendisi ayarlar; **`JAVA_HOME`** için kurulum sırasında ilgili seçeneği işaretleyin (*Set or override JAVA_HOME variable*)

Temurin - I



- Mesela Temurin JDK kurmak isterseniz, şöyle başlarsınız:

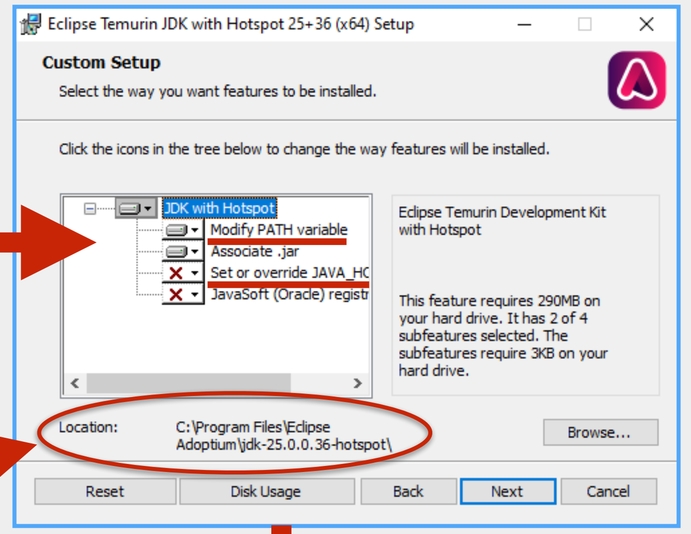


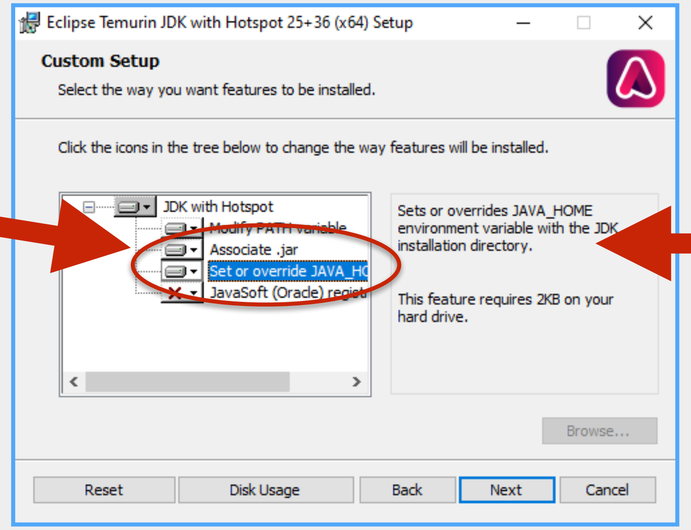
- Microsoft Build of OpenJDK kurulumu da çok benzerdir

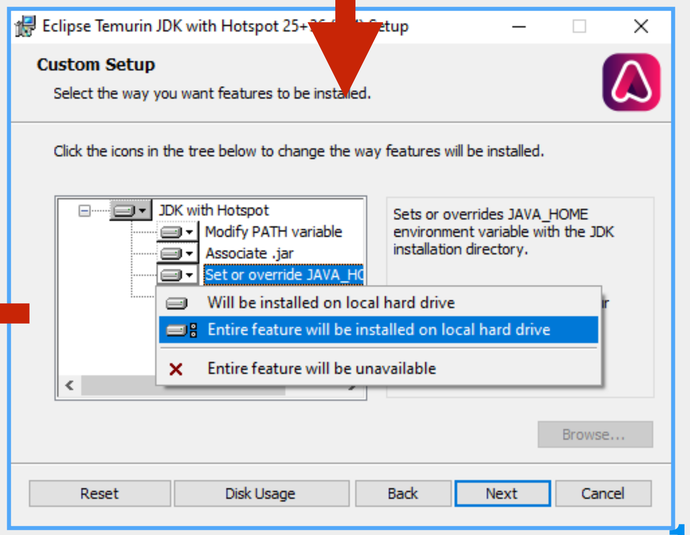
Temurin - II



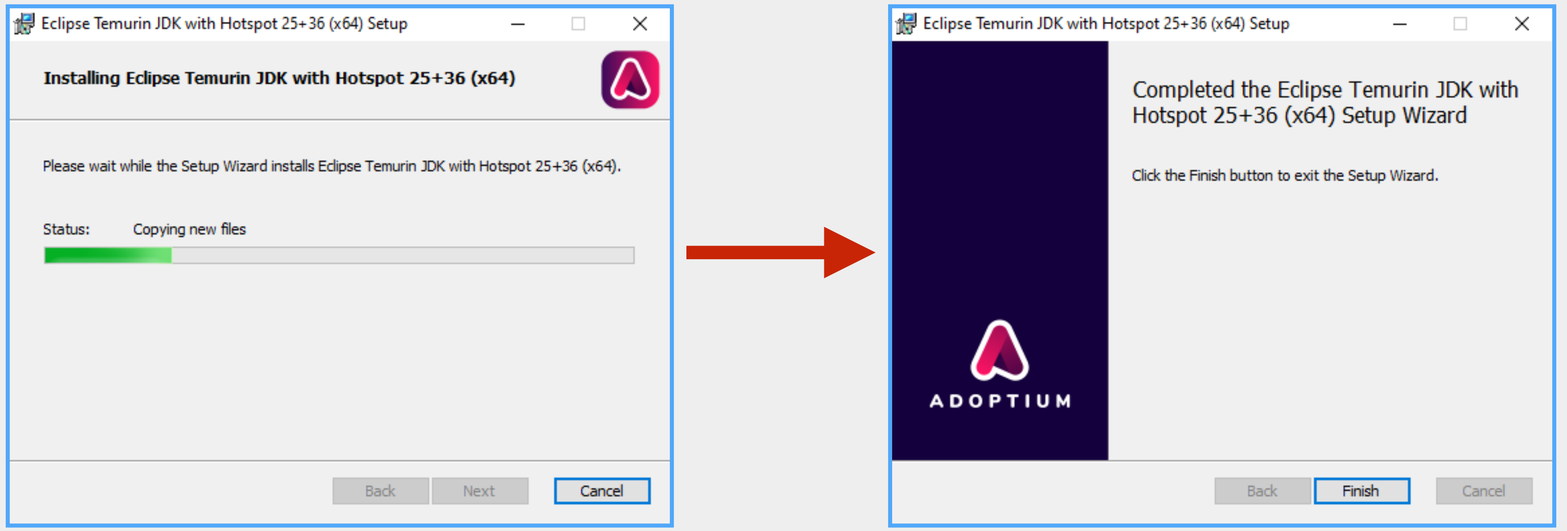
- Temurin kurulumu, **`PATH`** ayarını otomatik yapar ve **`JAVA_HOME`** için de size fikrinizi sorar
  - Kurulum dizini:
    - C:\Program Files\Eclipse Adoptium\jdk-25.0.0.36-hotspot
- **`JAVA_HOME`** için seçiminizi yaparsanız, kurulan JDK'ya göre değer atanır





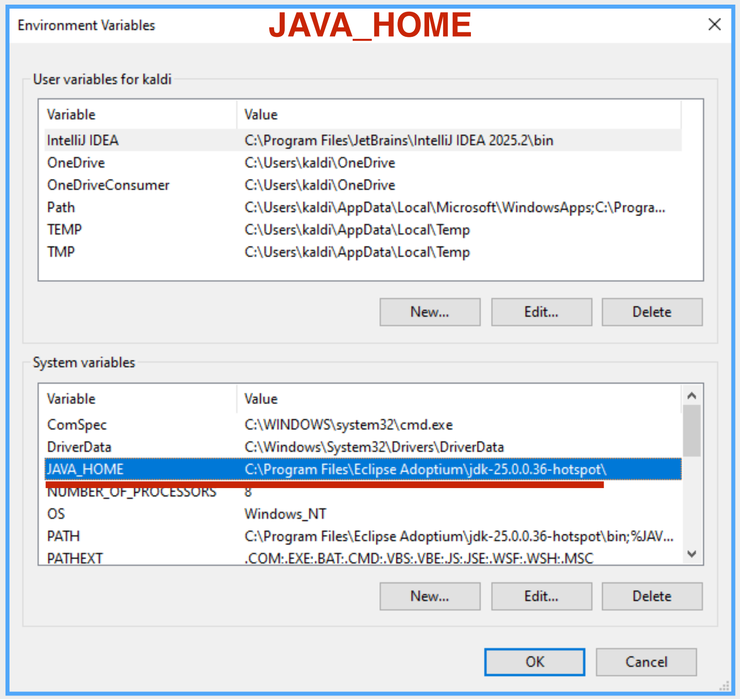


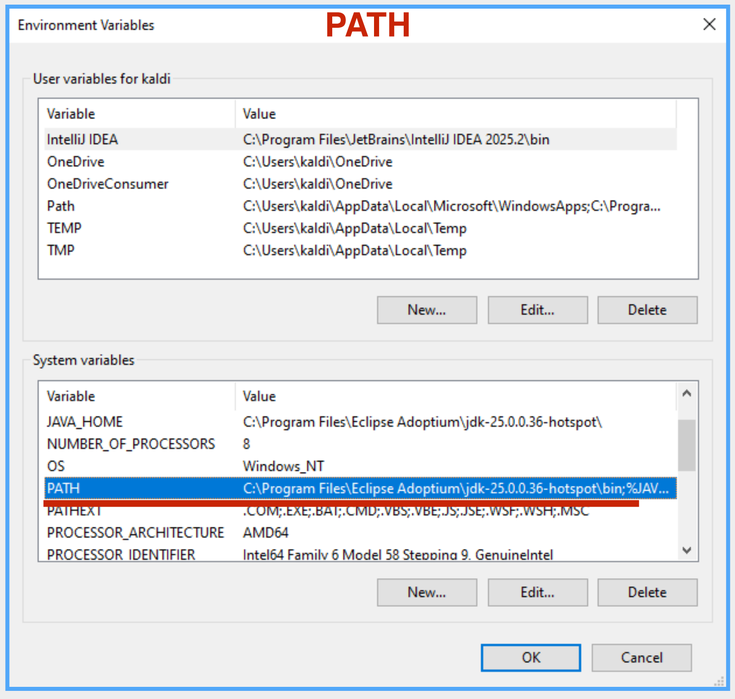
Temurin - III


Eclipse Temurin JDK with Hotspot 25+36 (x64) Setup
Installing Eclipse Temurin JDK with Hotspot 25+36 (x64)
Please wait while the Setup Wizard installs Eclipse Temurin JDK with Hotspot 25+36 (x64).
Status: Copying new files
Back
Next
Cancel
Eclipse Temurin JDK with Hotspot 25+36 (x64) Setup
ADOPTIUM
Completed the Eclipse Temurin JDK with Hotspot 25+36 (x64) Setup Wizard
Click the Finish button to exit the Setup Wizard.
Back
Finish
Cancel

Temurin - IV

- Bu şekilde JDK zahmetsizce kullanıma hazır olur
- Kurulum size bırakmadan **`PATH`** ve **`JAVA_HOME`** değerlerini uyumlu bir şekilde ayarlar





Ayarlardan Sonra - I



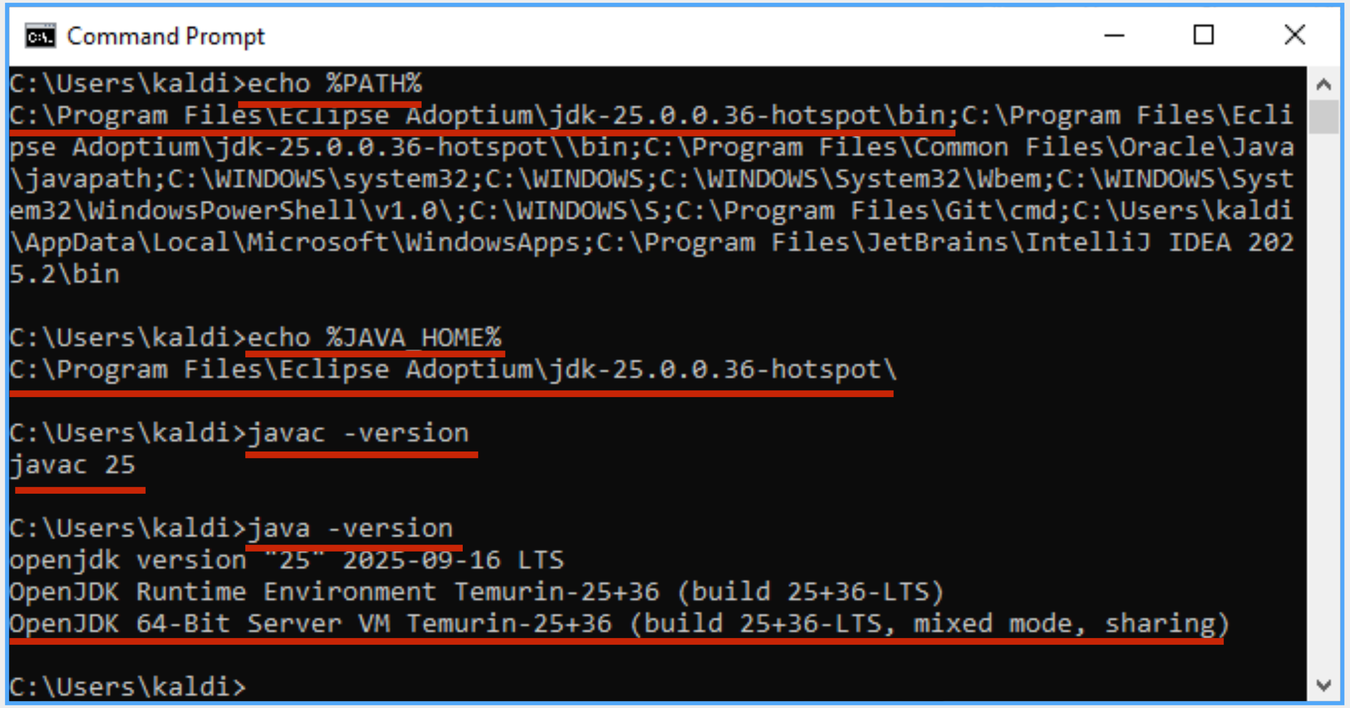
- Bu işlemlerden şu komutları çalıştırın:

```
echo %PATH%

echo %JAVA_HOME

javac -version

java -version
```



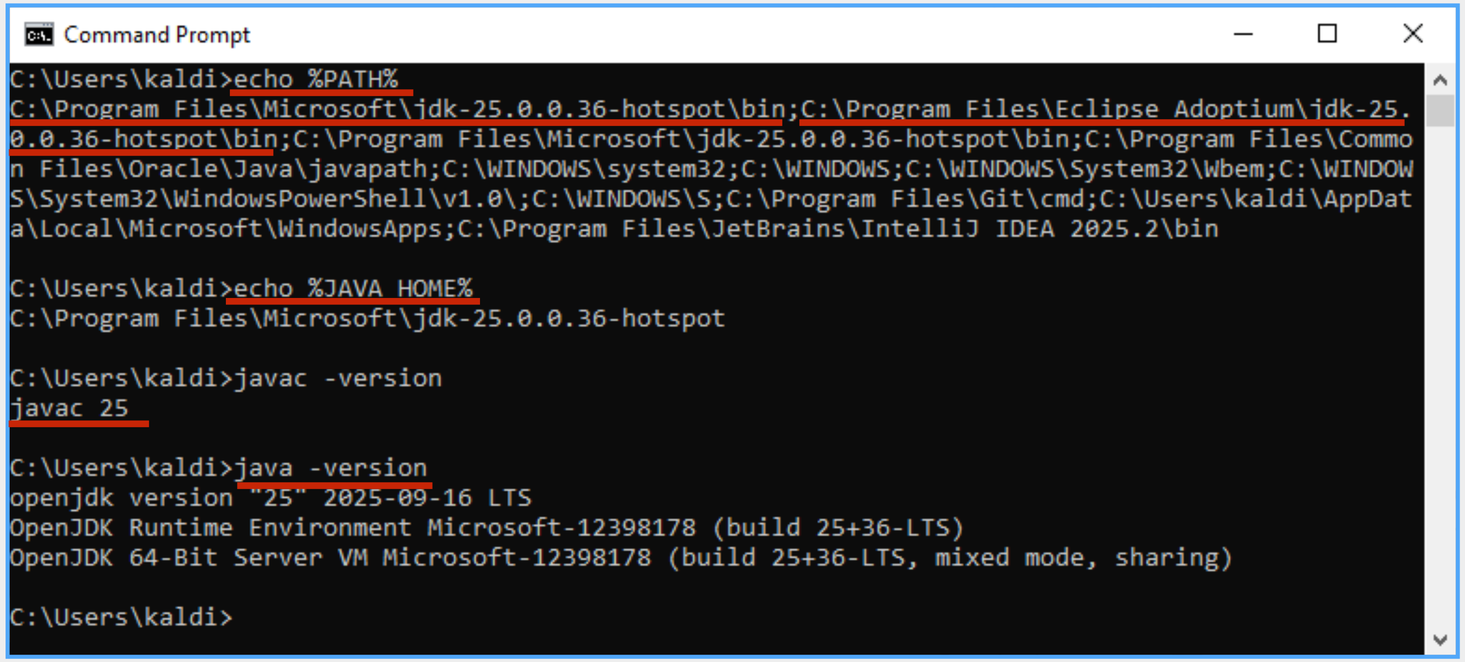
- Belirlenen **PATH** ve **JAVA_HOME** değerlerini komut satırında görebiliriz
- **javac** ve **java** komutları çalışır hale gelmiştir

Ayarlardan Sonra - II



- Eğer Microsoft Build of OpenJDK kurulursa benzer bilgileri bu sefer bu kurulum için komut satırında görürsünüz

- Hatta daha önce kurduğumuz Temurin, **`PATH`**'te ikinci sıradadır



Sonuç



- Windows kullanıcıları için JDK kurulumunda en basit çözüm Temurin ya da Microsoft Build of OpenJDK kullanmaktır
- Oracle JDK'da `JAVA_HOME`'u, Oracle'ın OpenJDK'sı gibi sıkıştırılmış kurulumlarda ise hem `PATH`'i hem `JAVA_HOME`'u daha önce açıkladığım gibi elle ayarlarsınız

Soru ve Cevap Zamanı!





macOS'ta JDK Kurulumu



macOS İçin JDK



- macOS'a da Oracle JDK, Oracle'ın OpenJDK'sı, Eclipse Temurin ya da Microsoft Build of OpenJDK kurulabilir
- Hangisini kurarsanız kurun JDK'lar `/Library/Java/JavaVirtualMachines` dizinine kurulur ve ayarlar shell üzerinden yapılır
- Bu yüzden önce Oracle JDK'yı kurup ayarlarını yapacağız; diğer dağıtımlar için de aynı adımlar geçerlidir

Oracle JDK



- Oracle JDK, macOS için `.dmg` olarak indirilir ve *next/continue* ile, varsayılan ayarlarla kurulur
- JDK 25 şu dizine kurulur: `/Library/Java/JavaVirtualMachines/jdk-25.jdk`
- JDK'nın dosyaları bu dizinin altındaki `Contents/Home`'dadır; `JAVA_HOME`'un göstermesi gereken yer burasıdır
- Güncelleme kurulumlarında dizinin adı değişebilir, örneğin *jdk-25.0.4.1.jdk*; bu yüzden patikayı elle yazmak yerine birazdan göreceğimiz `java_home` komutunu kullanacağız

Diğer JDK Dağıtımları



- macOS'a Eclipse Temurin, Microsoft Build of OpenJDK ya da bir başka JDK dağıtımı kurulabilir
- Bu dağıtımlar Homebrew paket yöneticisiyle tek komutla da kurulabilir:

```
brew install --cask temurin@25
```

```
brew install --cask microsoft-openjdk
```

Soru ve Cevap Zamanı!







macOS'ta PATH ve JAVA_HOME Ayarları

Shell



- macOS, Unix benzeri (Unix-like) bir işletim sistemidir; **`PATH`** ve **`JAVA_HOME`** gibi ayarlar shell ile, kullanıcı dizinindeki profil dosyasında yapılır
- macOS, Catalina'dan beri varsayılan shell olarak **ZSH** (**Z Shell**) kullanır; profil dosyası kullanıcı dizinindeki **`.zshrc`**'dir
- Hangi shell'i kullandığınızı şöyle görebilirsiniz: **`echo $SHELL`**
- **`.zshrc`**'yi bir editörde açmak için (dosya yoksa önce **`touch`** ile oluşturun):

```
touch ~/.zshrc
```

```
open -e ~/.zshrc
```

PATH ve JAVA_HOME - I



- `.zshrc` dosyasına şu iki satırı ekleyin:

```
export JAVA_HOME=$(/usr/libexec/java_home -v 25)

export PATH=$JAVA_HOME/bin:$PATH
```

- `/usr/libexec/java_home -v 25` komutu kurulu 25. sürüm JDK'nın patikasını verir; böylece dizin adını elle yazmanız gerekmez
- **`PATH`**, **`JAVA_HOME`**'dan türetildiği için ikisi hep aynı JDK'yı gösterir
- macOS'ta **`PATH`**'teki patikalar `:` ile ayrılır

PATH ve JAVA_HOME - II



- Değişikliğin etkisi için yeni bir terminal açın ya da şunu çalıştırın:

```
source  ~/.zshrc
```

Ayarlardan Sonra



- Bu işlemlerden sonra şu komutları çalıştırın:

```
echo $PATH

echo $JAVA_HOME

javac -version

java -version
```

```
akin@M3-Max:~
/Users/akin >echo $PATH
/opt/homebrew/opt/postgresql@16/bin:/Users/akin/.sdkman/candidates/gradle/curr
ent/bin:/Users/akin/.cabal/bin:/Users/akin/.ghcup/bin:/Users/akin/.rbenv/shims
:/Users/akin/.rbenv/bin:/Users/akin/bin:/usr/local/bin:/Users/akin/Commands:/o
pt/homebrew/bin:/Library/Frameworks/Python.framework/Versions/3.13/bin:/Librar
y/Frameworks/Python.framework/Versions/3.11/bin:/usr/local/bin:/System/Cryptex
es/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.crypt
exd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/
codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.syst
em/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin:/usr/local/share/dot
net:~/.dotnet/tools:/usr/local/go/bin:/Library/Frameworks/Mono.framework/Versi
ons/Current/Commands:/Applications/iTerm.app/Contents/Resources/utilities:/Use
rs/akin/.fig/bin:/Users/akin/.local/bin:/Users/akin/Library/Application Suppor
t/Coursier/bin:/Users/akin/Library/Application Support/JetBrains/Toolbox/scrip
ts:/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin:/Users/akin/
.lmstudio/bin
/Users/akin >echo $JAVA_HOME
/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home
/Users/akin >javac -version
javac 25
/Users/akin >java -version
openjdk version "25" 2025-09-16
OpenJDK Runtime Environment (build 25+36-3489)
OpenJDK 64-Bit Server VM (build 25+36-3489, mixed mode, sharing)
/Users/akin >
```

- `javac` ve `java` komutları çalışır hale gelmiştir

macOS'ta Bazı Komutlar



- macOS'ta `/usr/libexec/java_home -V` komutu kurulu bütün JDK'ları listeler
- Başka bir sürüme geçmek için `.zshrc`'deki -v 25'i örneğin -v 21 yapıp yeni bir terminal açmanız ya da `source ~/.zshrc` ile değişikliği etkinleştirmeniz yeter

Soru ve Cevap Zamanı!





Linux'ta JDK Kurulumu



OpenJDK'yı Kurmak - I



- Ubuntu gibi Debian tabanlı dağıtımlarda OpenJDK'yı kurmanın en kolay yolu paket yöneticisidir:

```
sudo apt install openjdk-25-jdk
```

- Sürümü bu şekilde adıyla isteyin; *default-jdk* dağıtımın varsayılan sürümünü kurar, o da 25 olmayabilir
- Adı *-headless* ile biten paketi kurmayın; o, Swing gibi grafik arayüz kütüphanelerini içermez

OpenJDK'yı Kurmak - II



- JDK'lar **`/usr/lib/jvm`** altına kurulur; hangilerinin kurulu olduğunu şöyle görebilirsiniz:

```
ls /usr/lib/jvm
```

- Kurulum ayrıca, **`/usr/bin`** altındaki **`javac`** ve **`java`** komutlarını yeni JDK'ya bağlar; **`/usr/bin`** zaten **`PATH`**'te olduğundan **`PATH`** ayarı gerekmez

- Bunu anlamak için komut satırında **`javac -version`** veya **`java -version`**'ı çalıştırabilirsiniz

Diğer JDK'ları Kurmak



- Oracle JDK, x64 için `.deb` paketi olarak indirilip şöyle kurulur; ARM64 için `.rpm` ya da sıkıştırılmış arşiv vardır:

```
sudo dpkg -i jdk-25_linux-x64_bin.deb
```

- Eclipse Temurin için önce Adoptium'un paket deposu eklenir, **sonra** `sudo apt install temurin-25-jdk` çalıştırılır: https://adoptium.net/installation/linux

- Sıkıştırılmış arşivle (tar.gz) kurulan JDK'larda **`PATH`** ve **`JAVA_HOME`**'u elle ayarlamanız gerekir

JAVA_HOME Ayarı



- **`JAVA_HOME`** kendiliğinden ayarlanmaz; kullanıcı dizinindeki **`.bashrc`** dosyasına şu satırı ekleyin:

```
export JAVA_HOME=/usr/lib/jvm/java-25-openjdk-amd64
```

- ARM64 işlemcili makinelerde dizinin adı **`java-25-openjdk-arm64`**'tür
- Değişikliğin etkisi için yeni bir terminal açın ya da **`source ~/.bashrc`** çalıştırın

Çoklu JDK Kullanımı



- Birden fazla JDK kuruluysa hangisinin kullanılacağını şu komutlarla seçersiniz:

```
sudo update-alternatives --config java

sudo update-alternatives --config javac
```

- İkisini ayrı ayrı seçin; yalnız **`java`**'yı değiştirmek **`javac`**'ı değiştirmez
- Bu seçim **`JAVA_HOME`**'u değiştirmez; **`PATH`** ile **`JAVA_HOME`**'un aynı JDK'yı göstermesi için **`.bashrc`**'deki **`JAVA_HOME`**'u da güncelleyin

Soru ve Cevap Zamanı!





Birden Fazla JDK



Birden Fazla JDK İle Çalışmak - I



- İş dünyasında çalışırken makinanızda birden fazla JDK kurulumunun olması çok tabidir
- Farklı projelerde farklı JDK'lar kullanılabilir
- IDE kullanırken bu durum IDE'ler üzerinden rahatlıkla hallolur; genelde IDE'ler her projenin JDK'sının ayarlanmasına izin verir
- Ama komut satırında iş yaparken, konfigürasyon/ayar dosyalarında vs. sıklıkla farklı JDK sürümleri arasında geçiş yapmanız gerekebilir

Birden Fazla JDK İle Çalışmak - II



- Bunu yönetmenin en basit şekli **`javac`** ya da **`java`** komutlarını mutlak patikalarıyla kullanmaktır
  - Örneğin Windows'ta komut satırında ya da bir konfigürasyon dosyasında 24. sürümün derleyicisini kullanmak için **`C:\Program Files\Eclipse Adoptium\jdk-24.0.2.12-hotspot\bin\javac`** denebilir
  - Bu şekilde **`PATH`** ve **`JAVA_HOME`** farklı bir kurulum gösterse bile komut satırında JDK 24 kullanılmış olacaktır

Birden Fazla JDK İle Çalışmak - III



- Benzer şekilde Ubuntu'da **`/usr/lib/jvm/java-25-openjdk-amd64/bin/java`** gibi mutlak patikayla istediğiniz JDK'daki komutu çalıştırabilirsiniz

Birden Fazla JDK İle Çalışmak - IV



- Bu mutlak patikalara *alias* vb. kısa yollar tanımlayabilirsiniz
  - Örneğin Windows'ta **`temurin24c.bat`** isimli bir dosyada yukarıdaki mutlak patika tanımlanabilir; bu şekilde **`temurin24c`** ilgili **`javac`** komutuna bir *alias* olur

Birden Fazla JDK İle Çalışmak - V



- Örneğin `.zshrc` dosyamdaki aliaslarla farklı JDK kurulumlarına kısa yollarla ulaşabildiğim gibi komut satırında `JAVA_HOME`'u da istediğim sürümü gösterecek şekilde değiştirebiliyorum

```
export JAVA_HOME=$(/usr/libexec/java_home -v 25)
export PATH=$JAVA_HOME/bin:$PATH
# Aliases to reach different JDK executables
alias javac17='$(/usr/libexec/java_home -v 17)/bin/javac'
alias java17='$(/usr/libexec/java_home -v 17)/bin/java'
alias javac21='$(/usr/libexec/java_home -v 21)/bin/javac'
alias java21='$(/usr/libexec/java_home -v 21)/bin/java'
# Aliases to set JAVA_HOME (and PATH) to different JDK versions
alias j17='export JAVA_HOME=$(/usr/libexec/java_home -v 17); export PATH=$JAVA_HOME/bin:$PATH; java -version'
alias j21='export JAVA_HOME=$(/usr/libexec/java_home -v 21); export PATH=$JAVA_HOME/bin:$PATH; java -version'
alias j25='export JAVA_HOME=$(/usr/libexec/java_home -v 25); export PATH=$JAVA_HOME/bin:$PATH; java -version'
```

Soru ve Cevap Zamanı!







Uygulama

İlk Kod



- İlk kodumuzu daha önce çalıştıramayıp sonra makinalarına başarılı bir şekilde JDK'ları kurup ayarlarını yapanlar, ilk kodumuzu yazdığımız **`Selam`** uygulamalarını şimdi çalıştırabilirler

```
class Selam{
   public static void main(String[] args){
    System.out.println("Selam :) ");
   }
}
```

```
void main() {
    System.out.println("Selam :) ");
}
```

```
void main() {
    IO.println("Selam :) ");
}
```

Selam24.java - I



- Şimdi makinanızda, masa üstünde (desktop) **`Code`** isimli bir yeni dizin yaratın ve içine **`ch01`** isimli başka bir dizin koyun
- Sonra aşağıdaki kodu kopyalayıp yapıştırarak ya da aynısını yazarak **`ch01`** dizininde **`Selam24.java`** isimli bir dosya yaratın

```
macOS ⇒ /Users/akin/Desktop/Code/ch01/Selam24.java
Windows ⇒ C:\Users\kaldi\Desktop\Code\ch01\Selam24.java
Ubuntu ⇒ /home/kaldiroglu/Desktop/Code/ch01/Selam24.java
```

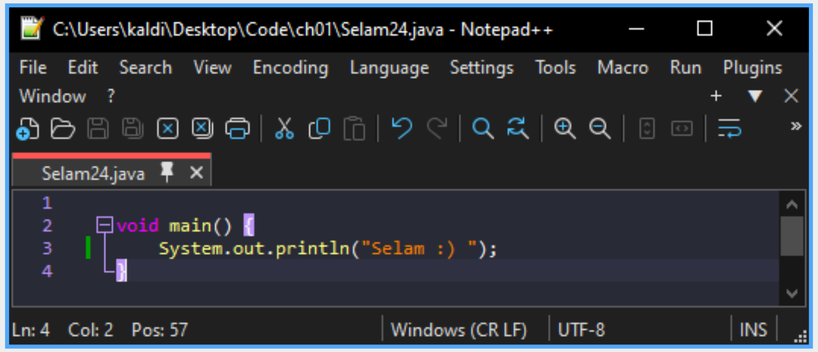
```
void main() {
    System.out.println("Selam :) ");
}
```

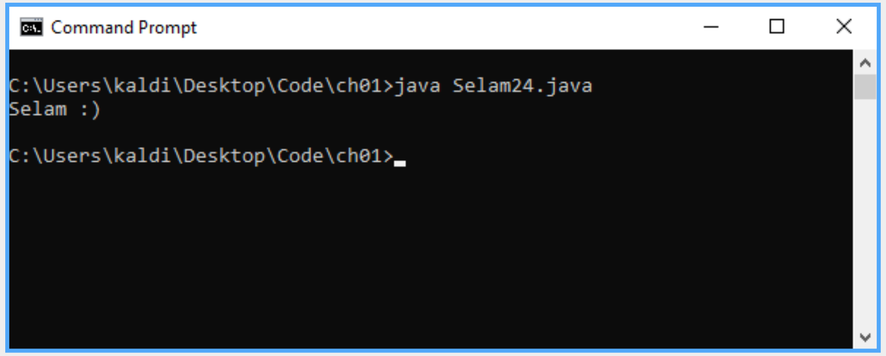
Selam24.java - II



- Daha sonra komut satırında **`ch01`** dizinine gidip **`java Selam24.java`** komutuyla ilk kodumuzu çalıştırın
- **`java`**, Java kodunu çalıştıran komuttur; bu komutla JVM, **`Selam24.java`** kodunu yükler ve çalıştırmaya başlar ve komut satırında **`Selam :)`**'ı görürüz

```
void main() {
    System.out.println("Selam :) ");
}
```





Selam25.java ve Selam.java



- Sonra aynı şeyi **`Selam25.java`** ve **`Selam.java`** için yapın

```
void main() {
    IO.println("Selam :) ");
}
```

```
class Selam{
   public static void main(String[] args){
    System.out.println("Selam :) ");
   }
}
```

- Bu kodları dersin başlarında da çalıştırmıştık, detaylarını görmek isteyenler oraya bakabilirler

SystemInfo - I



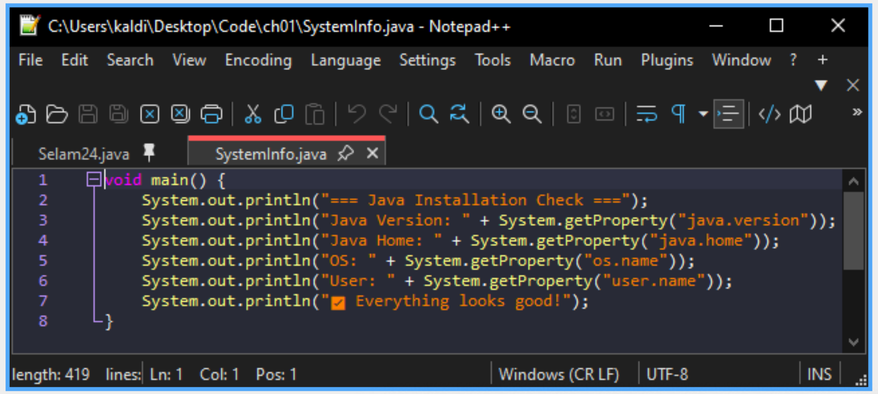
- Aşağıdaki kod parçasını **`SystemInfo.java`** isimli dosyaya yazın ve **`java SystemInfo.java`** komutuyla çalıştırın

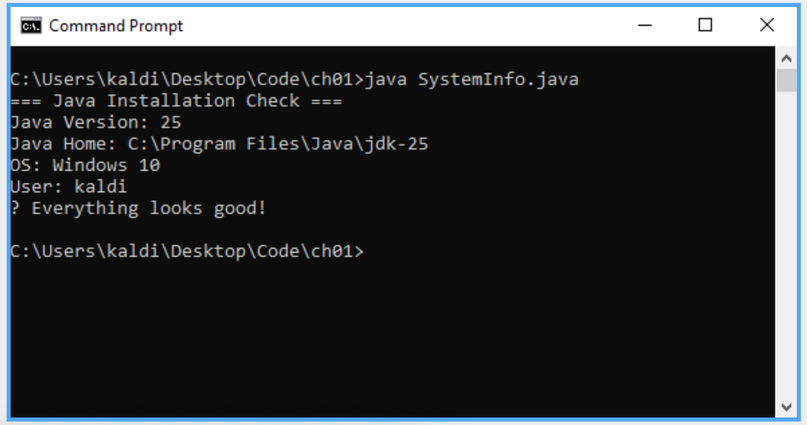
```
void main() {
    System.out.println("== Java Installation Check ==");
    System.out.println("Java Version: " + System.getProperty("java.version"));
    System.out.println("Java Home: " + System.getProperty("java.home"));
    System.out.println("OS: " + System.getProperty("os.name"));
    System.out.println("User: " + System.getProperty("user.name"));
    System.out.println("✅ Everything looks good!");
}
```

SystemInfo - II

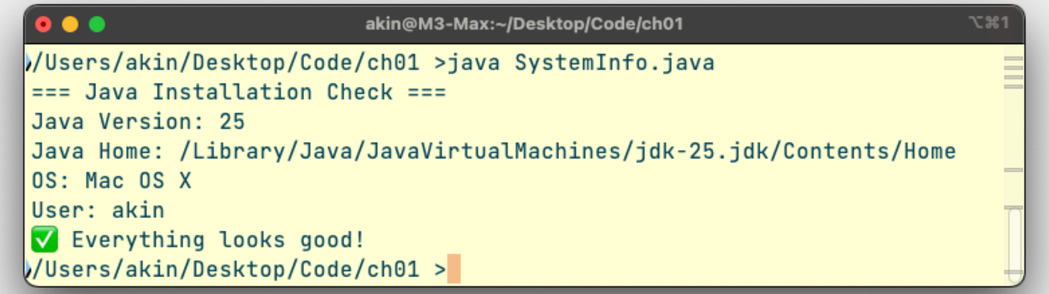


- Bu kod makinanızda erişilen ve **`SystemInfo.java`** isimli Java kodunu çalıştıran JDK sürümünü size verir





- Emojiler, Windows CMD'de bozuk görünebilir!



PatternPrinter



- Aşağıdaki kod parçasını **`PatternPrinter.java`** isimli bir dosyaya yazın ve **`java PatternPrinter.java`** komutuyla çalıştırın

```
void main() {
    for (int i = 1; i <= 10; i++) {
        for (int j = 1; j <= i; j++) {
            System.out.print("⭐ ");
        }
        System.out.println();
    }
    System.out.println("If you see stars, Java is installed correctly!");
}
```



Soru ve Cevap Zamanı!





JShell



JShell

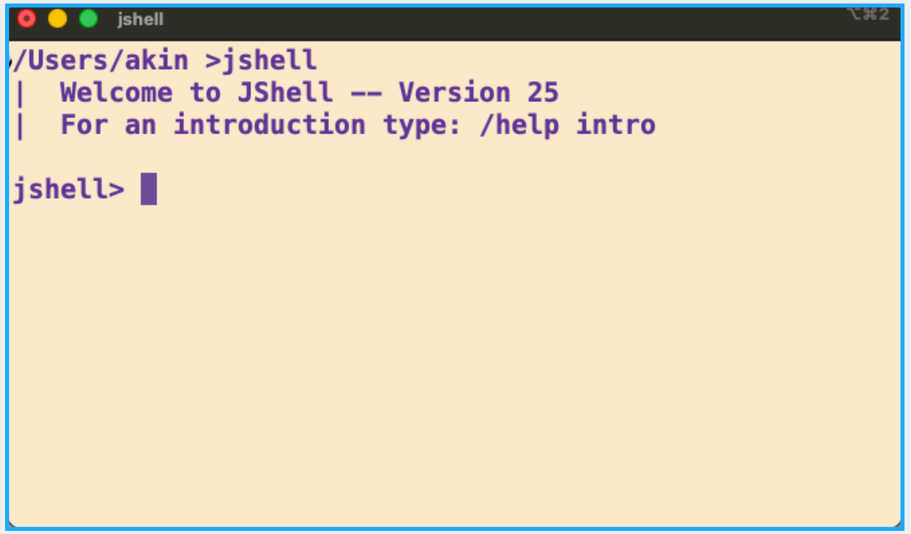


- **JShell**, Java'yı öğrenmeyi kolaylaştırmak amacıyla düşünülmüş bir komut satırı arayüzdür
  - Python gibi bazı dillerde çok eskiden bu yana vardır
- JShell bir **Read-Evaluate-Print Loop** (**REPL**) aracıdır, yani adım adım kod yazıp, çalıştırıp, sonucu görmeyi ve bu şekilde devam etmeyi sağlar
- İlk defa Java 9 ile birlikte gelmiştir

Başlatmak ve Bitirmek



- JShell, JDK kurulumunun **`bin`** dizinindeki **`jshell`** komutu ile başlatılır
- JShell başlarken varsayılan bir script çalıştırır, bununla en temel paketler import edilir
- JShell oturumu, **`/exit`** komutuyla da sonlandırılır



Yaz-Çalıştır



- Denemek amacıyla normalde **`main`** metot içine yazılıp derlenerek çalıştırılabilen tek satırlık kod JShell'de hızlıca çalıştırılabilir
- Satır sonlarına `;` koymaya da gerek yoktur

```
jshell                                                    ⌥⌘2
jshell> System.out.println("Selam!");
Selam!

jshell> System.out.println("Selam!")
Selam!

jshell> System.out.println("Selam!)
|  Error:
|  unclosed string literal
|  System.out.println("Selam!)
|                     ^

jshell> "Selam!"
$8 ==> "Selam!"

jshell> int i
i ==> 0

jshell> int j = 8
j ==> 8

jshell>
```

Değişken Tanımlama



- JShell'de değişken tanımlanabilir; değişkene doğrudan ulaşıldığında değeri basılır
- JShell bir değer üreten ve bir değişkene atanmayan her işlemin sonucunu, kendi oluşturduğu ve `$` ile başlayan isme sahip bir değişkene atar ve bunu paylaşır
  - Bu isimle o değişkene ulaşılmak mümkündür

```
jshell
jshell> int i = 5
i ==> 5

jshell> int j = 2 * i
j ==> 10

jshell> String s = "Selam!"
s ==> "Selam!"

jshell> s = s+ " Naber?"
s ==> "Selam! Naber?"

jshell> "Merhaba"
$15 ==> "Merhaba"

jshell> $15
$15 ==> "Merhaba"

jshell> Math.sqrt(2)
$17 ==> 1.4142135623730951

jshell> $17
$17 ==> 1.4142135623730951

jshell>
```

Metot Tanımlama ve Çağırma



- JShell'de metot tanımlanabilir
- Ayrıca metot çağrısı yapılabilir, metodun döndürdüğü değer alınabilir
- Java API'sinde halihazırda var olan metotlar yanında JShell'de tanımlanan metotlar da çağrılabilir

```
jshell
jshell> int add(int a, int b){
   ...>     return a + b;
   ...> }
|  created method add(int,int)

jshell> int c = add(2, 3)
c ==> 5

jshell> add(7, 9)
$21 ==> 16

jshell> $21
$21 ==> 16

jshell> double d = Math.sqrt(3)
d ==> 1.7320508075688772

jshell>
```

Tip Tanımlama



- JShell'de `class`, `interface`, `enum` vb. cinslerden tipler tanımlanabilir
- Tanımlanan tiplerin nesneleri oluşturulabilir
- Nesnelerin değişkenlerine ulaşılabilir ve metotları çağrılabilir

```
jshell
jshell> class Person{
   ...>     String name;
   ...>     int age;
   ...>
   ...>     public String toString(){
   ...>         return "Name: " + name + " Age: " + age;
   ...>     }
   ...> }
|  created class Person

jshell> Person p = new Person()
p ==> Name: null Age: 0

jshell> p.name = "Zeynep"
$3 ==> "Zeynep"

jshell> p.age = 25
$4 ==> 25

jshell> p
p ==> Name: Zeynep Age: 25

jshell>
```

JShell'i Kullanmak



- Java'yı öğrenirken küçük ve hızlı kod denemeleri için JShell'i kullanabilirsiniz
  - Örneğin, aklınıza takılan bir noktayı hızlıca JShell'de yazıp çalıştırabilirsiniz
- Bu gibi durumlarda JShell, IDE vb. kullanmaktan çok daha iyi bir yoldur

```
jshell
jshell> float f = 3.14
|  Error:
|  incompatible types: possible lossy conversion from double to
 float
|  float f = 3.14;
|            ^--^

jshell> float f = 3.14F
f ==> 3.14

jshell> double d = 3.14
d ==> 3.14

jshell> f == d
$8 ==> false

jshell> f < d
$9 ==> false

jshell> f > d
$10 ==> true

jshell> f - d
$11 ==> 1.0490417468034252E-7

jshell>
```

Kaynak



- Java Shell User Guide:
  - https://docs.oracle.com/en/java/javase/25/jshell/index.html
  - https://docs.oracle.com/en/java/javase/25/jshell/java-shell-user-guide.pdf

Soru ve Cevap Zamanı!







JShell Komutları

JShell Komutları



- JShell'e has komutlar `/` ile çalıştırılır
- Örneğin, **`/vars`**, **`/methods`** ve **`/types`**, shellde tanımlanmış değişkenler, metotlar ve tipler hakkında bilgi verir
- **`/list`** o ana kadar çalıştırılan tüm kodları listeler
- **`/list -all`** o ana kadar girilen tüm komutları, shell başlarken çalışan varsayılan kodlarla beraber listeler

Bazı Komutlar



- **`/<tab>`**: Var olan komutları listeler
- **`/!`**: Son çalıştırılan kodu bir daha çalıştırır
- **`/?`** ve **`/help`**: Yardımı basar
- **`CTRL-L`**: Komut satırını temizler
- Ayrıca JShell **`<tab>`** ile komut tamamlama özelliğine de sahiptir



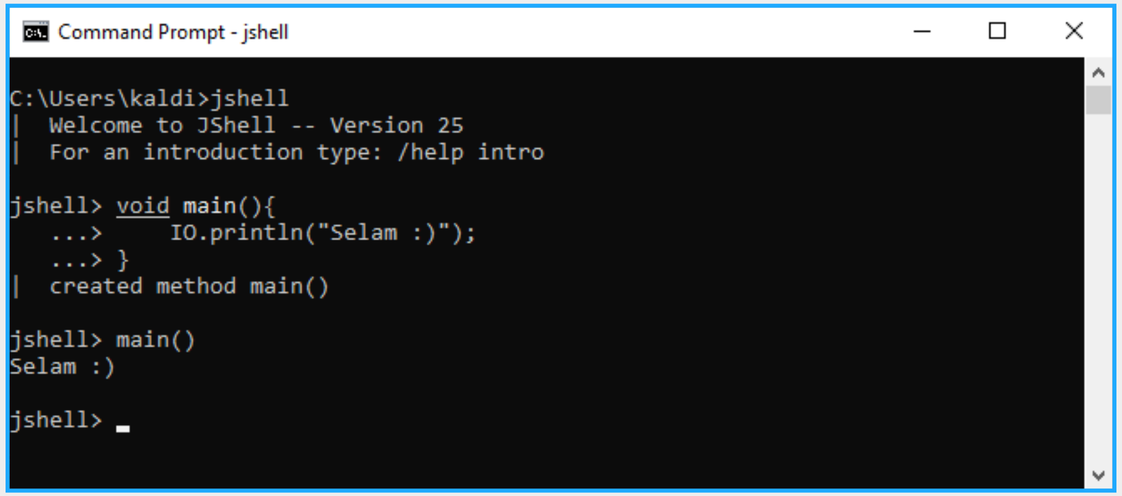
Uygulama

JShell'de İlk Kod



- Aşağıdaki kodu JShell'de yazıp sonrasında **`main()`** yazarak çalıştırın

```
void main() {
    IO.println("Selam :) ");
}
```



Soru ve Cevap Zamanı!





Geliştirme Ortamları



Java Geliştirme Ortamı



- Şu ana kadar gördüğümüz gibi Java kodu geliştirmek, derlemek, gerektiğinde debug etmek ve çalıştırmak için temelde JDK ile bir kelime işlemciye (editör) ihtiyaç vardır
- Fakat projelerdeki çok sayıda dosyayı yönetmek, komut satırından kurtulmak ve faydalı pek çok ek araç-gereçten (plug-in) faydalanmak için genelde IDE denilen gelişmiş yazılımlar kullanılır

Integrated Development Environment (IDE)



- **IDE**, **Integrated Development Environment**'ın (**Birleşik/Tümleşik Geliştirme Ortamı**) kısaltmasıdır
  - Yazılım geliştirme sürecinde, kod geliştirmeyi yönetmek için kullanılan araçtır
    - Kodun yazılması, derlenmesi, çalıştırılması, gerektiğinde debug edilmesi vb. işler genelde IDE'de yapılır
- Dolayısıyla IDE'ler, dillere özel yetkinliklere sahip kelime işlemci (editor) yanında pek çok farklı araca hatta artık AI'a da sahiptir

Java IDE'leri



- Java kodu geliştirmek için ücretli veya ücretsiz pek çok IDE vardır:
  - Eclipse www.eclipse.org
  - IntelliJ IDEA www.jetbrains.com/idea
  - Netbeans https://netbeans.apache.org/
  - Oracle JDeveloper https://www.oracle.com/application-development/technologies/jdeveloper.html
  - MS Visual Studio Code: https://code.visualstudio.com/ with Oracle's Java Platform Support extension

Derste Kullanım



- Bu derste isteyen istediği IDE'yi kullanabilir
- Ben, **IntelliJ IDEA** kullanıyorum
- Zaman zaman, az önce yaptığımız gibi, bir IDE olmaksızın, basit bir kelime işlemci (editor) ile kod yazıp, komut satırını kullanarak kodu derleyip çalıştıracağız
- Şimdi size kısaca IntelliJ IDEA'yı tanıtacağım



IntelliJ IDEA

IntelliJ IDEA



- IntelliJ IDEA'yı, kısaca IntelliJ: https://www.jetbrains.com/idea/
- 2025.3'ten itibaren tek bir IntelliJ IDEA indirilir; Ultimate ve Community artık ayrı ürünler değildir
- Eski Community Edition'ın bütün özellikleri, ticari kullanım dahil, ücretsizdir
- Ultimate özellikleri ise ücretli abonelik ister; öğrenci e-postasıyla kayıt olunduğunda bir sene ücretsiz kullanılabilir, lisans her sene yenilenmelidir
- Bu derste ücretsiz özellikler yeterlidir

IntelliJ ve AI



- IntelliJ ile pek çok AI aracını kullanabilirsiniz
- IntelliJ'in ücretli kendi **AI Assistant**'ı var; bununla Claude, Gemini, ChatGPT ve diğer modelleri kullanabilirsiniz
- Github hesabınızı bağlarsanız **Copilot**'ı da bedava kullanabilirsiniz

Soru ve Cevap Zamanı!







Dersin Kaynak Kodları

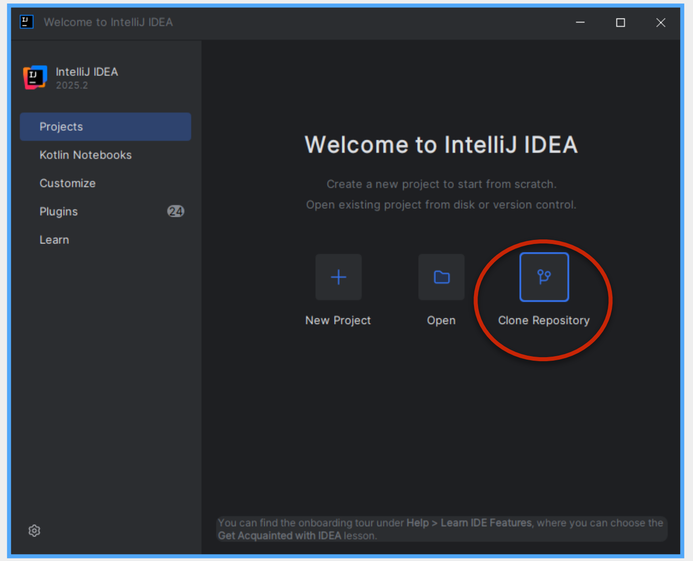
Kaynak Kodlar



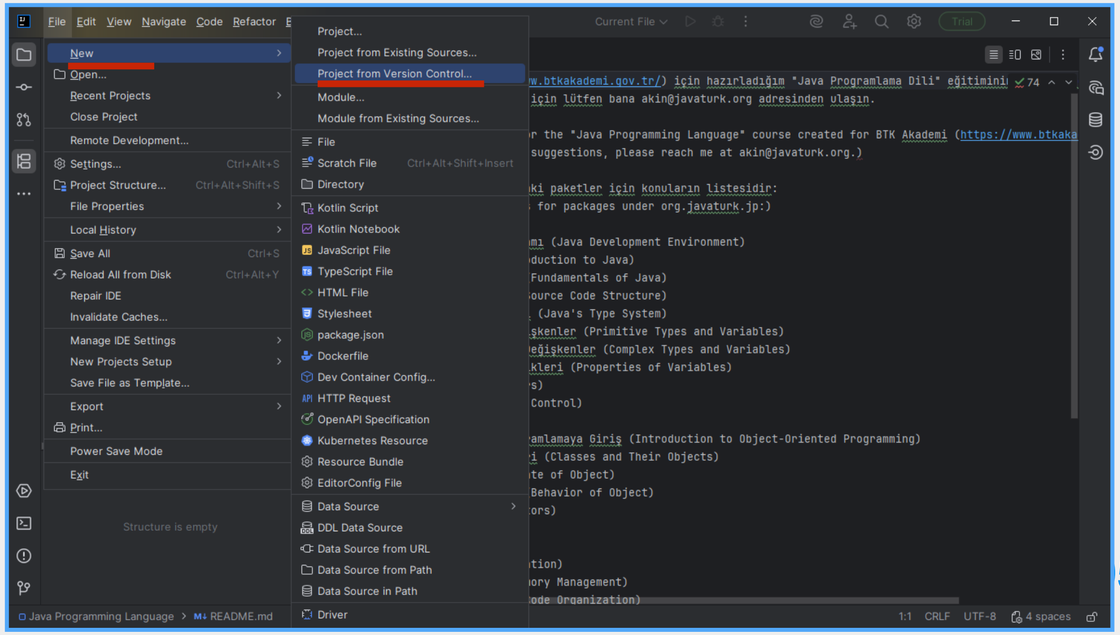
- Bu dersin kaynak kodlarına benim Github hesabımdaki repodan (repository) ulaşabilirsiniz
  - https://github.com/kaldiroglu/JavaProgrammingLanguage
- IntelliJ'i kurup çalıştırdıktan sonra ilk yapılacak şey dersin kaynak kodlarını makinanıza indirmektir; buna **cloning a repo** da denir
- Bu repoya zaman zaman değişiklikler yapabilirim, bunu takip edip güncellemeleri makinanıza alabilirsiniz

Kaynak Kodları İndirmek - I

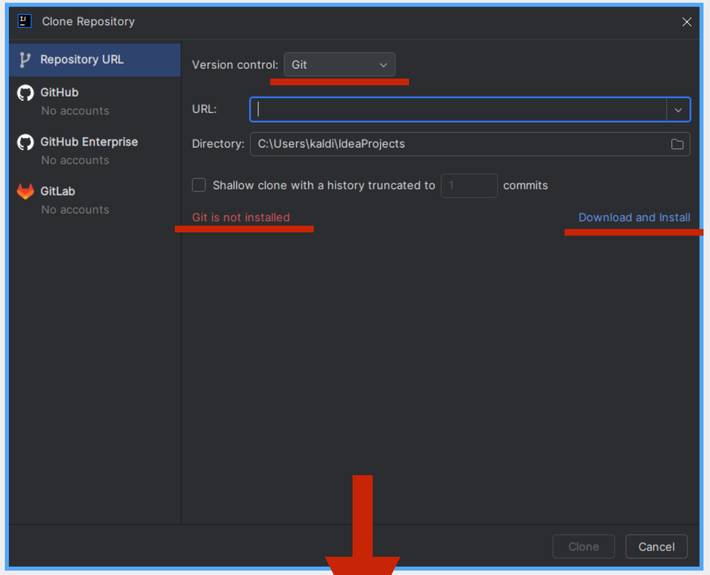
- Bu amaçla, IntelliJ'i ilk defa çalıştırdığınızda karşınıza çıkan pencerede **`Clone Repository`**'yi tıklayın

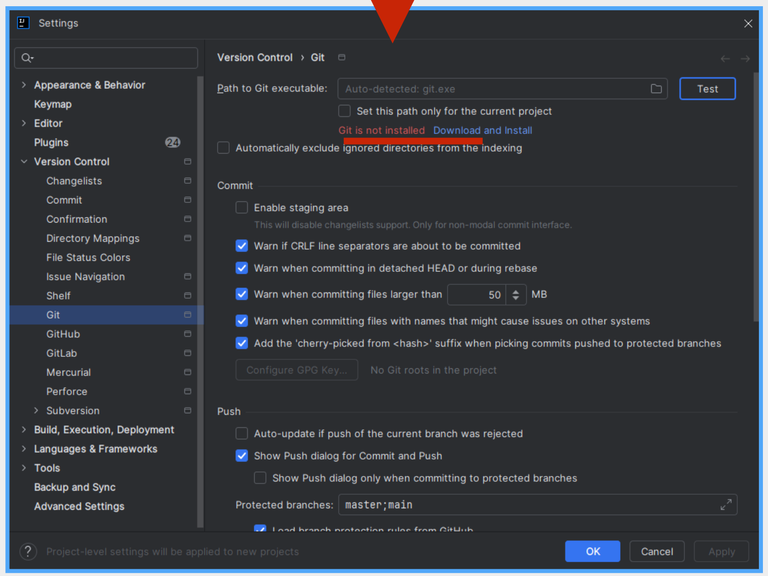


- IntelliJ zaten önceden açık ise **`File -> New`** ya da **`New Project`** tıklanır ve sonrasında **`Project from Version Control`** seçilir



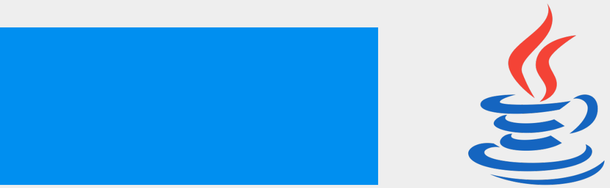
Kaynak Kodları İndirmek - II





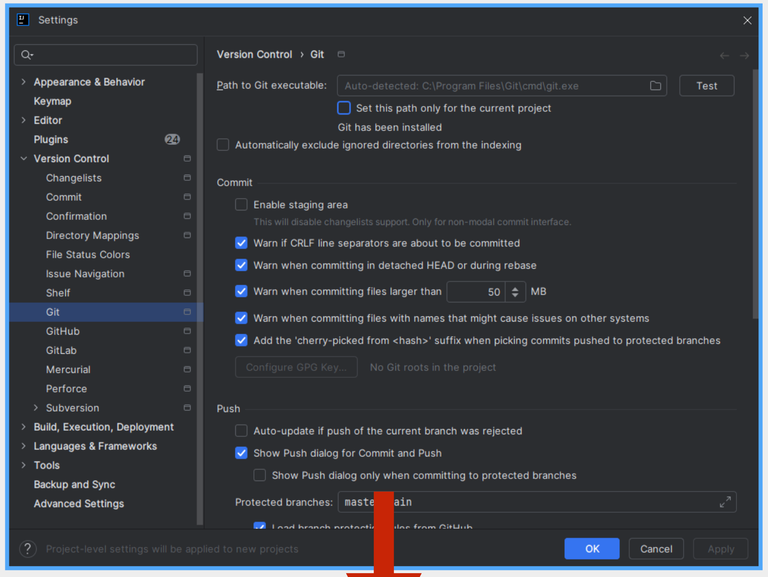
- Çıkan pencerede eğer daha önce kurulmadıysa IntelliJ, Git'i kurmak isteyecektir, kurun
  - Ubuntu'da komut satırında **`sudo apt-get install git-all`** ile kurabilirsiniz
- Windows ve macOS'ta zaten açık pencerede sizi kurmak için yönlendirir
- Git, macOS'ta **`brew install git`** ile de kurulabilir

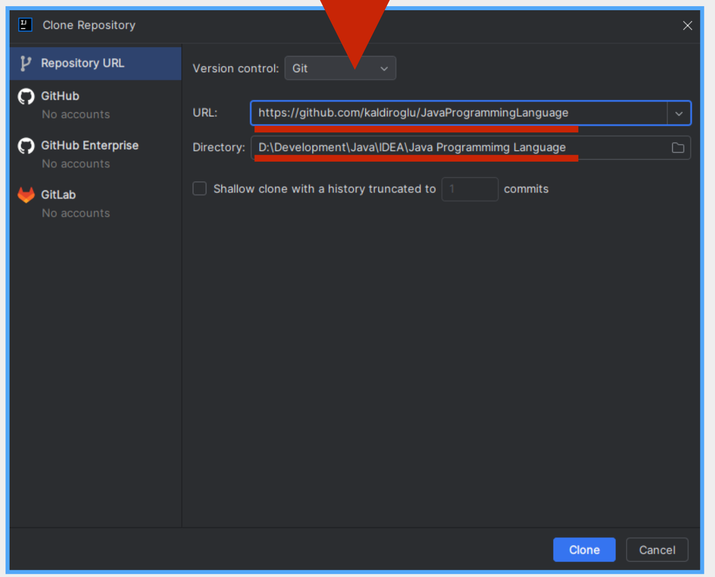
Kaynak Kodları İndirmek - III



- Sonra **`Repository URL`** seçiliyken **`URL:`**'e Github repo adresini ve **`Directory:`**'e projeyi indireceğiniz dizini girin ve **`Clone`**'u tıklayın

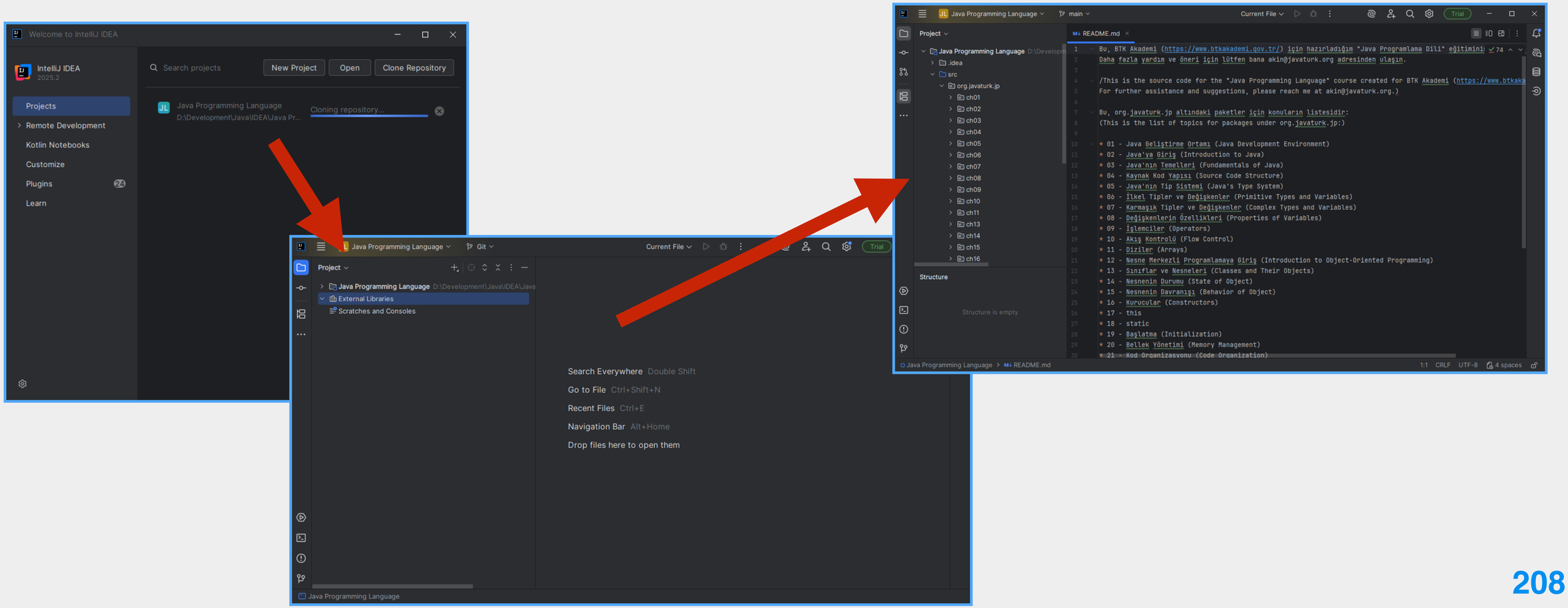
- IntelliJ, Github'daki repodan proje kodlarını makinanıza indirmeye başlayacaktır





Kaynak Kodları İndirmek - IV

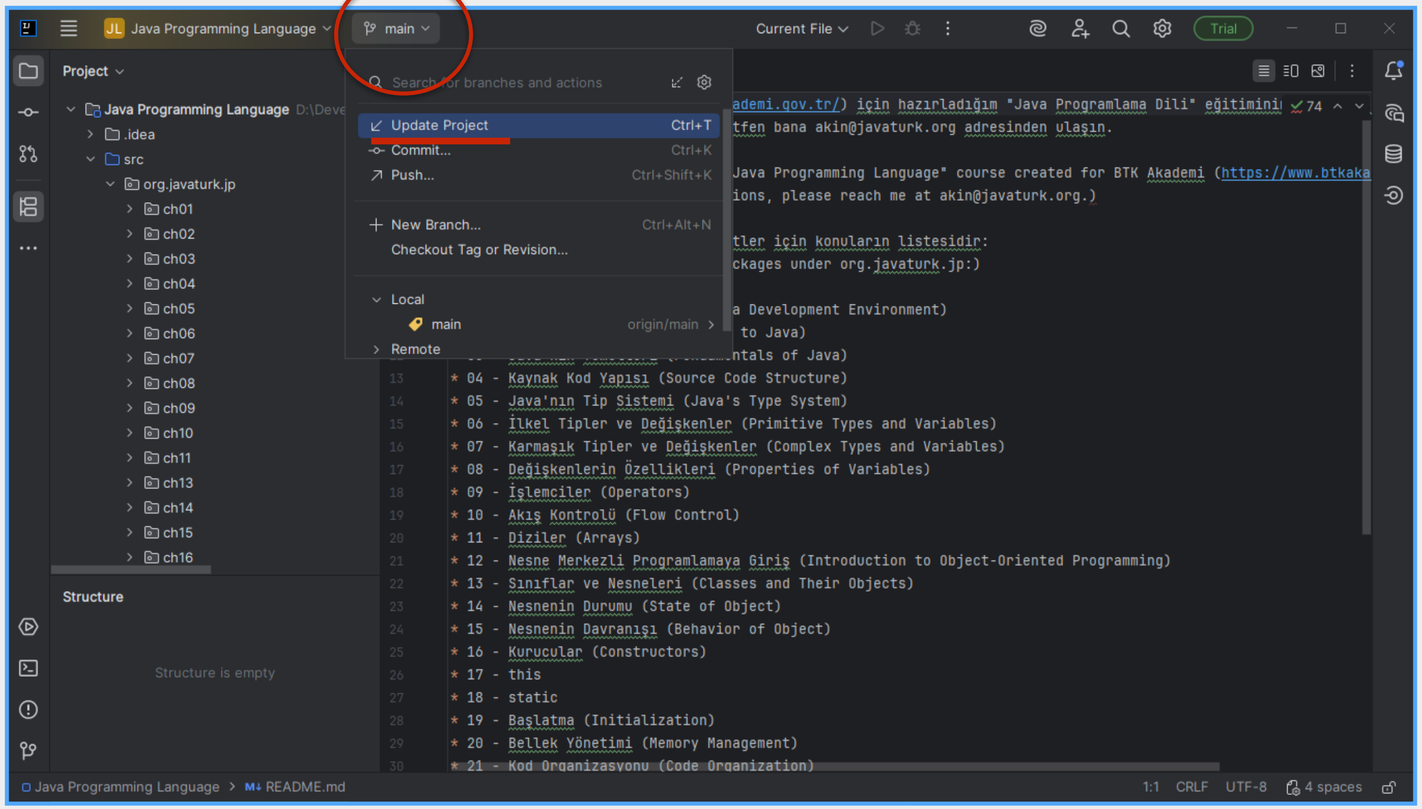
- İndirme bitince IntelliJ, **Java Programming Language** projesini makinanızda hazır hale getirecektir



Kaynak Kodları İndirmek - V



- Sonrasında zaman zaman projeyi güncelleyip varsa Github'tan değişiklikleri indirebilirsiniz



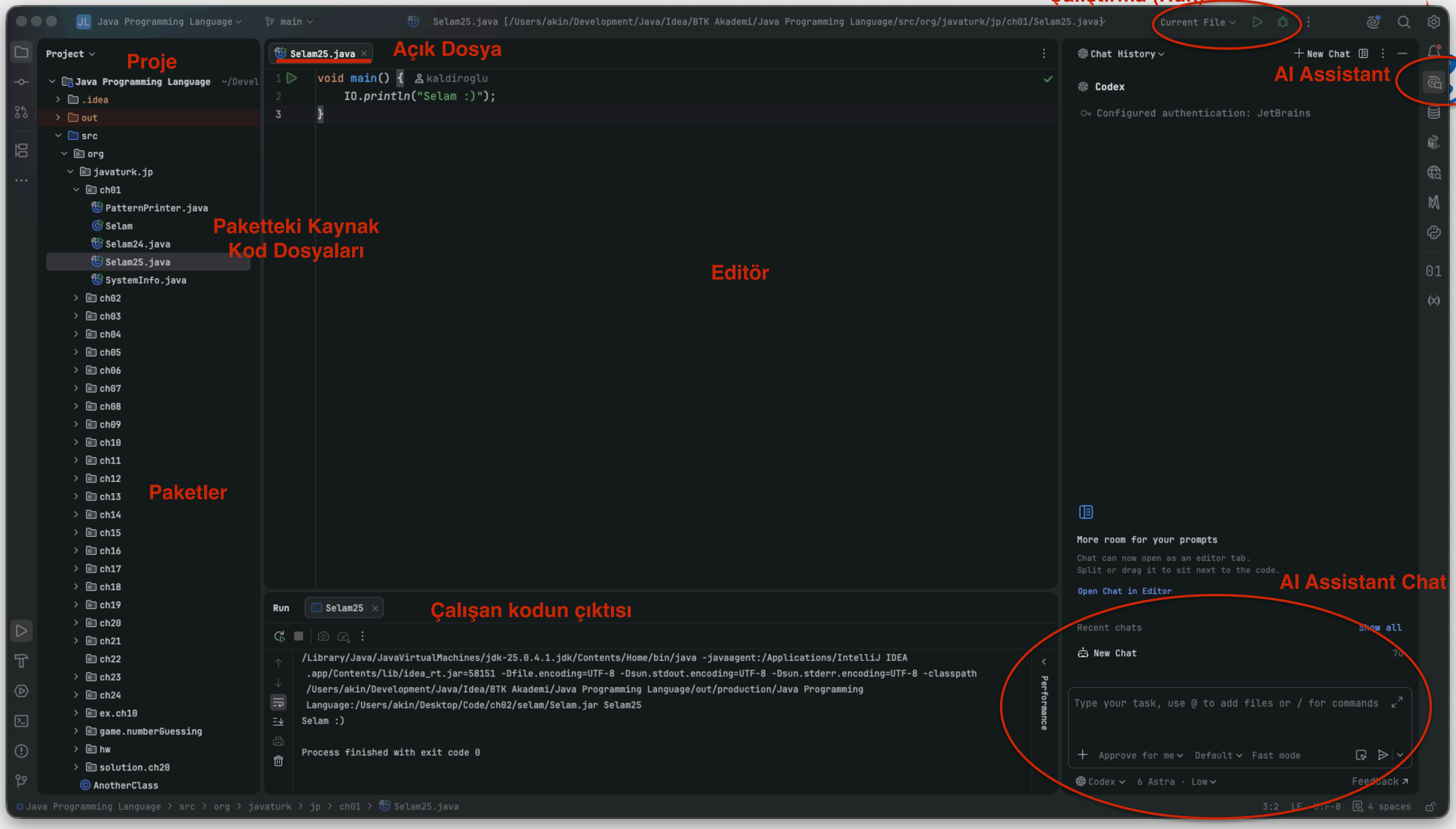


Uygulama

IDE'de İlk Kod



- IntelliJ'le **Java Programming Language** projesini makinanıza indirdiğinize göre artık az önce komut satırında çalıştırdığımız kodları orada da çalıştırabiliriz
- Projenin altındaki `src`'yi tıkladığınızda paketleri göreceksiniz
- `org.javaturk.jp`, kök pakettir (root package) ve tüm bölümlerin örnek kodları, kökün altındaki `chXX` paketlerindedir
  - Örneğin `Selam` uygulamaları `org.javaturk.jp.ch01` paketindedir


Çalıştırma (Run)
Java Programming Language
main
Selam25.java [/Users/akin/Development/Java/Idea/BTK Akademi/Java Programming Language/src/org/javaturk/jp/ch01/Selam25.java]
Current File
Project
Proje
Java Programming Language ~/Devel
.idea
out
src
org
javaturk.jp
ch01
PatternPrinter.java
Selam
Selam24.java
Selam25.java
SystemInfo.java
ch02
ch03
ch04
ch05
ch06
ch07
ch08
ch09
ch10
ch11
ch12
ch13
ch14
ch15
ch16
ch17
ch18
ch19
ch20
ch21
ch22
ch23
ch24
ex.ch10
game.numberGuessing
hw
solution.ch20
AnotherClass
Paketteki Kaynak Kod Dosyaları
Paketler
Selam25.java
Açık Dosya
1 void main() { kaldiroglu
2 IO.println("Selam :)");
3 }
Editör
Run
Selam25
Çalışan kodun çıktısı
/Library/Java/JavaVirtualMachines/jdk-25.0.4.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA
.app/Contents/lib/idea_rt.jar=58151 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
/Users/akin/Development/Java/Idea/BTK Akademi/Java Programming Language/out/production/Java Programming
Language:/Users/akin/Desktop/Code/ch02/selam/Selam.jar Selam25
Selam :)
Process finished with exit code 0
Performance
Chat History
New Chat
AI Assistant
Codex
Configured authentication: JetBrains
More room for your prompts
Chat can now open as an editor tab.
Split or drag it to sit next to the code.
Open Chat in Editor
AI Assistant Chat
Recent chats
Show all
New Chat
Type your task, use @ to add files or / for commands
Approve for me
Default
Fast mode
Codex
6 Astra · Low
Feedback
Java Programming Language > src > org > javaturk > jp > ch01 > Selam25.java
3:2
4 spaces

Soru ve Cevap Zamanı!







Yeni Proje Oluşturma

Yeni Proje Oluşturma



- Bu derse çalışırken kendi kodlarınızı yazıp çalıştırabileceğiniz, mesela **`My Java Works`** isimli bir projeniz olmalı
- Şimdi bu projeyi IntelliJ'de oluşturup ilk kodumuzu yazalım
- Yeni bir proje oluşturmak için **`File -> New -> Project`** ile açılan pencerede solda Java seçili olsun ve **`Name`** için **`My Java Works`** girin ve **`Location`** için de Java projelerinizin dizininin patikasını girin ve sağ alttaki **`Create`** düğmesini tıklayın

Proje Özellikleri



- Proje üzerinde sağ tık yapıp **`Open Module Settings`**'i seçtiğinizde **`Project Structure`**'a ulaşırsınız
- Burada solda **`Project`**'i seçip projenin hangi JDK'yı kullandığını, dil seviyesini (language level), yeni özellikleri kullanıp kullanmayacağını vs. belirleyebilirsiniz
- İlerleyen derslerde yeri geldikçe IntelliJ'in farklı özelliklerini de göstereceğim

Soru ve Cevap Zamanı!





Ödevler



Ödev



- JDK 25'in ve IDE'nin kurulup, dersin kaynak kodlarının IDE üzerinden çalıştırılabilir hale getirilip, **`Selam25.java`**, **`Selam24.java`** ve **`Selam.java`**'nın hem komut satırında hem de IDE üzerinden başarıyla çalıştırılması bu bölümün tek ödevidir

Bundan Sonra?



Bundan Sonra?



- Bu bölümde Java geliştirme ortamını, JDK, JRE, JVM gibi en temel öğelerini, komutlarını, kabaca nasıl kullanılacaklarını, hatta *çalışma zamanı* gibi bazı kavramlarını vs. öğrendik
- Şimdi Java dilini öğrenmeye hazırız
- Bundan sonraki bölümde Java'ya giriş yapacağız, en temel özelliklerini öğrenmeye başlayacağız

Bölüm Sonu

Soru ve Cevap Zamanı!



